

Bitsliced Segment-Based Search Technique for Low-Depth and Hardware-Efficient S-Box Circuits

Giyeon Kim^{*1}, Seungjun Baek^{*2}, Yongjin Jeon³, Vedad Hadžić⁴, and
Jongsung Kim^{5,6}(✉)

¹ Department of IT Convergence and Application Engineering, Pukyong National University,
Republic of Korea

gykim@pknu.ac.kr

² Department of Financial Information Security, Kookmin University, Republic of Korea

hellosj3@kookmin.ac.kr

³ COSIC, KU Leuven, Belgium

idealtop18@gmail.com

⁴ Intel Corporation, Austria

vedad.hadzic@intel.com

⁵ Department of Cyber Security, Kookmin University, Republic of Korea

jskim@kookmin.ac.kr

⁶ Department of Information Security, Cryptology, and Mathematics, Kookmin University,
Republic of Korea

Abstract. In this paper, we propose a new widely applicable technique for constructing low-depth S-box circuits, which we call **SLICE** (*Segmented Low-depth Iterative Circuit Exploration*). **SLICE** reduces circuit depth by partitioning circuits into subcircuits and applying bit-level optimizations. To mitigate the optimization cost of subcircuits with large bit-widths or high AND depth, **SLICE** temporarily reduces their bit-width during the search, making low-depth circuit construction feasible for various S-box sizes. Furthermore, we refine the eBPD algorithm with the aim of minimizing XOR gate count in terms of area, and apply it to the constructed circuits to achieve an additional reduction.

Our proposed method is simple yet powerful, especially in practical applications. This work focuses on three practically deployed cases, namely the **AES**, **Ascon** S-box circuits and Dillon’s 6-bit APN S-box used in **FIDES**, and additionally considers the cube x^3 S-box. For the **AES** S-box, we present 14-, 13- and 12-depth circuits, whereas the previous lowest depth was 14. Notably, the 12-depth design sets a new overall depth record for **AES** S-box circuits, while the 13-depth design records the most hardware-efficient circuit in terms of the area $\times$ delay metric (ADP). For the **Ascon** S-box, we present the first 4-depth circuit. For Dillon’s S-box and the cube x^3 S-box, we also derive new low-depth circuits that improve upon the best previously known depths. All of our proposed S-box circuits achieve better ADPs than previous designs. We believe that **SLICE** will be useful for evaluating both existing and novel S-box designs.

Keywords: AES · Ascon · Dillon · Cube · S-box · Low-latency · Logic optimization · SLICE

^{*}These authors contributed equally to this work.

1 Introduction

Lightweight cryptography is gaining increasing traction across modern deployments. In ultra-low-latency 5G use cases such as remote surgery, smart factories, and smart grids, as well as driver-assistance systems (ADAS), cipher latency can directly impact end-to-end timing, which has renewed interest in shallow circuit realizations with small critical-path delay. At the same time, cryptographic primitives are widely deployed in resource-constrained environments such as low-end Internet of Things devices, where tight area constraints make the area-latency trade-off a central design concern. Consequently, optimizing a single metric in isolation, such as gate count or circuit depth, may not yield the best overall implementations in practice; instead, area and latency should be considered jointly.

NIST’s internationally standardized block cipher AES has long been a prime target for circuit-level optimization. Efforts have focused on minimizing the XOR gate count in implementing its linear matrix over $\mathbb{F}_2$, reducing the gate equivalent cost. This task is known as the shortest linear program (SLP) problem, which is NP-complete [BMP08] and must be addressed via heuristics. Boyar and Peralta [BP10] introduced the BP algorithm, a distance vector minimizing heuristic that iteratively updates a base set to approach the target matrix. Subsequent refinements include random-normal-BP (RNB) [TP20] and BP with a depth limit (BPD) [LSL⁺19], and various other proposals [VSP18, XZL⁺20, LXXZ21, SFX23, LZW23].

Beyond linear matrix optimizations, AES employs a heavy S-box, making the optimization of its bitsliced implementation crucial. Most efforts directly or indirectly rely on Canright’s $\mathbb{F}_{2^8}$ inverter [Can05], which recursively constructs operations from smaller subfields down to $\mathbb{F}_2$, where additions, multiplications, and inverses become XOR gates, AND gates, and identity operations, respectively. Jeon et al. [JBKK25] recently adapted the BP algorithm—originally for linear layers—to handle nonlinear operators in S-box circuits. They used a circuit rewriting technique to modify XOR gate dependencies and integrated an automated nonlinear gate addition process into the BP algorithm. These enhanced algorithms, extended RNB (eRNB) and BPD (eBPD), have been applied to optimize the S-box circuits of AES [DR02], SNOW3G [For06], and Saturnin [CDL⁺20], demonstrating reductions in XOR count and/or depth. Unlike SAT-based solvers [Sto16], LIGHTER [JPST17], and graph-based optimizers [JBK24], these algorithms operate directly on circuit descriptions rather than S-box truth tables and natively support the AES S-box. Hadžić and Bloem [HB25] proposed a modified Canright’s inverter structure that increases the XOR gate count but achieves an AND depth of 3 and a circuit depth of 14 for the AES S-box circuit, currently the lowest known depth.

Ascon, designed by Dobraunig et al. [DEMS21], is a cryptographic primitive featuring permutation-based AEAD and hashing schemes. NIST selected it as the lightweight cryptography standard in February 2023, and it was officially standardized in August 2025 [STM⁺25]. Ascon uses a 5-bit S-box with circuit depth 5 and AND depth 1, a depth record that remains unbroken. Dillon’s S-box is a 6-bit APN permutation [BDMW10] used in the lightweight AEAD scheme FIDES [BBK⁺13]. Prior work describes how to generate Dillon’s S-box [PUB16], but does not provide an explicit gate-level circuit, and thus does not report the depth of a Boolean implementation. Another compact target is the cube x^3 S-box over $\mathbb{F}_{2^n}$; for $n \in \{6, 7\}$, prior ad-hoc implementations reach depth 7.

Contributions. In this paper, we propose SLICE, a novel technique for constructing low-depth S-box circuits. Combined with complementary optimizations, SLICE yields new circuits for AES, Ascon, Dillon, and the cube x^3 S-box, achieving record-low depths while maintaining competitive gate counts. Our contributions are summarized as follows:

1. Novel Technique for Generating Low-Depth S-Box Circuits (SLICE).

Global optimization of an S-box circuit requires a clear understanding of the interactions among its components, which becomes increasingly challenging as the bit-width or degree of each bit grows. We tackle this issue in part by dividing the circuit by AND depth, a process called *slicing*, which reduces the depth of each subcircuit individually. In order to reduce the depth of each part, we define a node pool that collects constructible nodes. They initially correspond to elements from a bitsliced representation of the identity permutation, and organize these pools into a chain. We then construct the entire circuit by combining the bitsliced representations of each subcircuit, obtained by *slicing* the target S-box, using the chain of node pools. To handle the high computational cost caused by subcircuits with large bit-width or high AND depth, we employ a *segmenting* technique. In this approach, the node bit-widths are temporarily reduced to explore candidate subcircuits and are restored to the original size to complete the entire circuit. The proposed technique is broadly applicable to low-depth S-box constructions: for small-bitwidth S-boxes, *slicing* alone is often sufficient, whereas larger S-boxes may also require *segmenting*.

2. Novel Compact AES, Ascon, Dillon, and Cube x^3 S-Box Circuits.

We apply the SLICE technique to derive low-depth implementations of several AES S-box circuits, improving on prior methods. By optimizing Canright’s structure [Can05], we discover a novel 14-depth circuit that is best among AND depth 4 designs in terms of depth as well as AND and XOR gate counts. For Hadžić and Bloem’s structure [HB25], we obtain the first 12- and 13-depth circuits, best among AND depth 3 designs; the 12-depth design sets a new depth record for AES, while the 13-depth design further reduces the XOR gate count compared with prior work. For the Ascon S-box [DEMS21], SLICE achieves the first depth reduction, from 5 to 4; due to its small bit-width, *slicing* alone is sufficient. We also apply SLICE to Dillon’s 6-bit APN S-box [BDMW10] and to the 6- and 7-bit cube x^3 S-boxes, yielding new low-depth circuits with additional gate-count improvements. We further reduce XOR gates using refined eBPD [JBKK25]. Table 1 compares our results to prior work.

We synthesized the S-box circuits in Table 1 as 2-fanin designs with the Nangate 45nm Open Cell Library, and evaluated their ASIC performance using OpenSTA. During synthesis, we replaced existing AND and XOR gates with more efficient realizations such as NAND and XNOR, without any increase in the counts of nonlinear and linear gates, the AND depth, or the overall depth. To ensure fairness, we adopt the $\text{area} \times \text{delay}$ (ADP) metric as in [RTA18], which better reflects implementation efficiency than gate count or depth alone because different gate types incur different area and delay costs. Based on this metric, we found that our 13-depth AES S-box circuit outperforms previously reported designs, achieving at least a 13% improvement in ADP regardless of the underlying structure. For Ascon S-box, our 4-depth circuit improves ADP by at least a 30% over [DEMS21]. Similarly, for Dillon’s 6-bit APN S-box and the 6- and 7-bit cube x^3 S-box, ADP is reduced by at least 54%, 61%, and 70%, respectively, over prior designs. To support realistic masked-hardware design points, we evaluate bit-level masked implementations derived from our proposed circuits and report their randomness consumption and area, showing competitive overhead under side-channel protection.

Organization. Sect. 2 introduces the AES, Ascon, Dillon, and the cube x^3 S-box circuits. Sect. 3 describes SLICE technique for constructing low-depth S-box circuits. Sect. 4 applies SLICE to derive low-depth and hardware-efficient implementations of the AES and Ascon S-box circuits, as well as Dillon S-box and the cube x^3 S-box in the 6- and 7-bit settings.

Table 1: Comparison of results for AES, Ascon, Dillon, and the cube x^3 S-box circuits.

Size	S-box Circuit	$\mathcal{D}$	$\mathcal{AD}$	$\#\mathcal{NL}$	$\#\mathcal{L}$	$\#\mathcal{G}$	Area (GE)	Delay (ns)	ADP	Reference
8	AES	28	6	32	83	115	198	1.55	306.9	[BP10]
		27	6	32	81	113	194	1.49	289.06	[CMT16]
		26	6	32	81	113	194	1.4	271.6	[JBKK25]
		21	5	50	69	119	192	1.4	268.8	[RTA18]
		17	5	50	79	129	212	0.9	190.8	[RTA18]
		17	5	32	93	125	218	0.96	209.28	[JBKK25]
		16	4	34	94	128	233.33	1.01	225.57	[BP12]
		16	4	34	91	125	217.33	1.01	219.51	[BFP19]
		16	4	33	101	134	235	0.93	218.55	[JBKK25]
		15	4	34	100	134	235.33	0.92	216.51	[JBKK25]
		14	4	33	95	128	223	0.84	187.32	This paper (List. 1)
		14	3	42	97	139	236	0.85	200.6	[HB25]
		13	3	42	94	136	230	0.72	165.6	This paper (List. 2)
		12	3	42	107	149	256	0.68	174.08	This paper (List. 3)
7	x^3	8	1	15	80	95	175.66	0.47	82.56	[BDMD ⁺ 20]
		-	1	11	45	56	-	-	-	[BDM25] [‡]
		6	1	11	41	52	93	0.37	34.41	This paper (List. 7)
		5	1	18	40	58	104	0.24	24.9	This paper (List. 8)
		5	1	20	37	57	97.66	0.25	24.42	This paper (List. 9)
6	Dillon	14	3	12	57	69	124	0.88	109.12	[PUB16] [†]
		10	3	12	39	51	91.33	0.54	49.32	This paper (List. 5)
		7	2	20	49	69	120	0.43	51.6	This paper (List. 6)
	x^3	7	1	9	43	52	95.33	0.39	37.18	[BDMD ⁺ 20]
		-	1	8	37	45	-	-	-	[BDM25] [‡]
		6	1	8	28	36	64	0.37	23.68	This paper (List. 10)
		5	1	9	27	36	63.33	0.28	17.73	This paper (List. 11)
		4	1	11	23	34	58	0.25	14.5	This paper (List. 12)
	Ascon	5	1	5	11	16	31	0.23	7.13	[DEMS21]
		4	1	5	11	16	29	0.17	4.93	This paper (List. 4)

$\mathcal{D}$: depth / $\mathcal{AD}$: AND depth / $\#\mathcal{NL}$: nonlinear gate count / $\#\mathcal{L}$: linear gate count / $\#\mathcal{G}$: total gate count / ADP: Area-Delay product.

[†] This work describes how to generate the Dillon S-box; however, we could not reproduce the instance from the described algorithm. Instead, we derived a new instance from the proposed structure with minor refinements.

[‡] This work reports only $\#\mathcal{L}$ and $\#\mathcal{NL}$, without providing circuits; thus, $\mathcal{D}$ cannot be determined.

Sect. 5 refines the eBPD algorithm to reduce the XOR gate count of circuits generated using SLICE and evaluates the cost of hardware bit-level masking using a customized Compress tool. Finally, Sect. 6 concludes the paper.

2 Background

2.1 Definitions of Circuit Metrics

Throughout the paper, we denote the following metrics for a circuit: the circuit depth $\mathcal{D}$, the AND depth $\mathcal{AD}$, the nonlinear gate count $\#\mathcal{NL}$, the linear gate count $\#\mathcal{L}$, and the total gate count $\#\mathcal{G}(=\#\mathcal{NL}+\#\mathcal{L})$. We consider only 2-fanin gates in this paper.¹ Here, $\mathcal{D}$ denotes the length of the longest path from input to output, and $\mathcal{AD}$ refers to the maximum number of AND gates on paths over the basis (AND, XOR, NOT). We sometimes refer to circuits with AND depth 1 and 2 as $1\mathcal{AD}$ and $2\mathcal{AD}$. $\#\mathcal{NL}$ counts nonlinear gates (AND, OR, NAND, NOR), while $\#\mathcal{L}$ counts linear gates (XOR, XNOR).

¹This setting is standard in prior work and keeps our metrics directly comparable; it also ensures portability across arbitrary netlist libraries and technology-mapping flows. If a target library offers wider primitives (e.g., 3-input XORs), our circuits can be translated by a simple post-processing script that repacks trees of 2-input gates into the corresponding multi-input cells.

2.2 Brief Descriptions of AES, Ascon, Dillon, and the Cube x^3 S-Boxes

For **AES**, we consider the two representative circuits of Canright [Can05] and Hadžić–Bloem [HB25]. We use the specification circuit for **Ascon** [DEMS21], and refer to [BDMW10, PUB16] for the definitions of **Dillon**. Further background on these S-boxes and the baseline circuits used in this work is provided in Appendix A.

AES. Both Canright’s and Hadžić and Bloem’s circuits for the **AES** S-box are based on a tower field representation of $GF(2^8)$, where inversion is efficiently computed using algebraic transformations over $GF(2^4)$, $GF(2^2)$, and $GF(2)$. Canright’s circuit computes the multiplicative inverse in the 8-bit finite field by systematically deriving intermediate values, which ultimately produce the final inverse result. Hadžić and Bloem refine this structure by focusing on the shared multiplicative component in both output branches. They simplify the computation by reorganizing the multiplication over a smaller subfield basis, enabling reuse of intermediate results and partial parallelization. This modification reduces the total AND depth while preserving the original algebraic structure.

Ascon. **Ascon**, designed by Dobraunig et al. [DEMS21], is a cryptographic primitive that includes permutation-based AEAD and hashing schemes. The **Ascon** S-box is a 5-bit bijective nonlinear function with algebraic degree 2, differential and linear branch number 3, and no fixed points. Due to its bitsliced structure and low degree, it enables efficient masked implementations resistant to side-channel attacks.

Dillon and x^3 . Dillon et al.’s 6-bit APN permutation [BDMW10], which we call **Dillon** S-box, is the only known APN permutation for even input sizes and has been used in the authenticated cipher FIDES. Perrin et al. [PUB16] reverse-engineered its structure and presented a simple decomposition based on the cube function over $\mathbb{F}_{2^3}$ (a special case of the butterfly structure), enabling efficient bitsliced and hardware implementations. However, these works do not provide an explicit gate-level circuit for the full **Dillon** S-box. The cube x^3 S-box is the quadratic power mapping $x \mapsto x^3$ over $\mathbb{F}_{2^n}$, where field elements are represented using an irreducible polynomial basis. Being quadratic, it naturally admits $1\mathcal{AD}$ implementations, and it is bijective for $n = 7$ but not for $n = 6$.

3 SLICE: New Segment-Based Technique for Generating Low-Depth S-Box Circuits

In this section, we present **SLICE** (*Segmented Low-depth Iterative Circuit Exploration*), a technique for generating low-depth S-box circuits.

3.1 Overview of SLICE Technique

Before introducing the formal notation and the detailed algorithms, we first summarize the overall workflow of **SLICE**. Starting from a baseline gate-level implementation of an S-box, **SLICE** decomposes the optimization task into smaller subcircuits, searches for low-depth circuits of these subcircuits, and then combines them into a full circuit. Figure 1 illustrates this workflow. The dashed box in Figure 1 represents the core **SLICE** pipeline, whereas the final post-processing step in this work applies eBPD [JBKK25] with additional refinements.

1. **Starting from a baseline S-box circuit.** **SLICE** takes as input a baseline gate-level circuit of the target S-box. This baseline may come either from a previously known implementation or from a circuit synthesized from the S-box truth table.

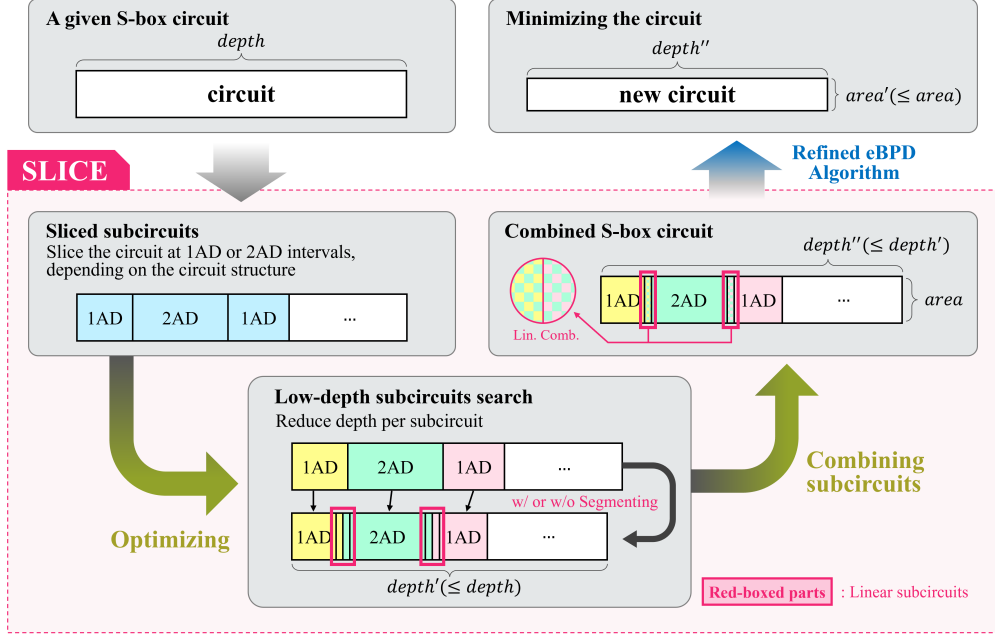


Figure 1: Proposed method for constructing low-depth S-box circuits.

2. **Slicing the circuit into AD -bounded subcircuits.** The baseline circuit is first partitioned into subcircuits bounded by AND depth, typically at $1AD$ or $2AD$ intervals, depending on the circuit structure.
3. **Searching for low-depth subcircuits.** Each sliced subcircuit is then optimized independently by a forward/backward search over node-pool chains. The goal is to generate low-depth candidates for each output bit and then recover concrete gate-level realizations from the recorded split/decode data. When direct exploration at the original bit-width becomes infeasible, *segmenting* temporarily compresses the bitsliced representation, performs the search in the reduced space, and then lifts valid candidates back to the original space through merging and filtering. The search-space limitation, low-depth search, and segmenting procedures are described in Sects. 3.3–3.5.
4. **Recombining the optimized subcircuits into a full S-box circuit.** After candidate circuits are obtained for the sliced subcircuits, the outputs of each slice are completed into a full subcircuit while favoring minimal depth and nonlinear gates. The optimized slices are then stitched back together to reconstruct the full S-box circuit. When two adjacent slices end and begin with linear regions, these regions can be merged across the slice boundary; in Figure 1, such linear regions are highlighted by the red boxes. This merging step can further reduce the overall depth without increasing area. This step is described in Sects. 3.6 and 3.7.
5. **Further minimizing the resulting circuit.** Finally, we apply refined eBPD as a post-processing step to the combined S-box circuit. This step further reduces the remaining XOR count while preserving the low depth achieved by SLICE, thereby improving the final hardware cost. Our refinements to eBPD are described in Sect. 5.

The remainder of this section develops the core SLICE pipeline in detail, while the refined eBPD post-processing step is presented separately in Sect. 5.

3.2 Notation and Terminology

$\mathbb{F}_2$ is the finite field with the set $\{0, 1\}$ equipped with multiplication $\cdot$ and addition $\oplus$ operations, and $\mathbb{F}_2^n$ is an n -dimensional vector space over $\mathbb{F}_2$. A *gate* is a function from $\mathbb{F}_2^*$ to $\mathbb{F}_2$, and depending on the value of $*$, it is referred to as a $*$ -fanin gate. For example, 2-fanin gates such as the XOR and AND gates are defined for $a, b \in \mathbb{F}_2$ as $XOR(a, b) = a \oplus b$ and $AND(a, b) = a \cdot b$, respectively. The circuit of an S-box S is defined as a sequence of gates and nodes, where a *node* refers to either an input or an output of a gate. In the circuit of S , we primarily consider XOR and AND gates, as other linear and nonlinear operators can be implemented by combining them with NOT gates.

We introduce the *bitsliced* technique, which represents an n -bit S-box S as an n -tuple of coordinate functions $S = (s_{n-1}, \dots, s_0)$, where each Boolean function $s_i : GF(2^n) \rightarrow GF(2)$ is encoded as a 2^n -bit truth table vector [JPST17]. For example, the identity permutation over $GF(2^3)$ and $GF(2^4)$ is given by the tuples $(0x0F, 0x33, 0x55)$ and $(0x00FF, 0x0F0F, 0x3333, 0x5555)$, respectively. This representation applies a Boolean instruction to all input values at once; for instance, $AND(s_1, s_0) = AND(0x3333, 0x5555) = 0x1111$. Here, although the input domain is $GF(2^4)$, the AND operation is interpreted bitwise, multiplying each pair of corresponding bits independently.

A collection of nodes in bitsliced representation is called a *node pool*. For an n -bit input, the *initial node pool* is defined as

$$\mathcal{N}^{(n,0)} = \{0, i_0^n, i_1^n, \dots, i_{n-1}^n\},$$

where each $i_k^n \in \mathbb{F}_{2^n}$ is obtained from the bitsliced representation of the identity permutation I . We omit the superscript n when the bit-width is clear, and include 0 in $\mathcal{N}^{(n,0)}$ for convenience in computations.

We denote by $\mathcal{N}_*^{(n,k)}$ the node pool obtained from $\mathcal{N}^{(n,0)}$ after applying k gate layers. The subscript $*$ $\in \{L, X, A\}$ indicates the last operation used: L for all linear combinations (arbitrary XORs), X for one XOR layer, and A for one AND layer. We omit $*$ when clear from context. The nodes are formally defined as follows:

Definition 1 (Node Pool). For the n -bit initial node pool $\mathcal{N}^{(n,0)}$, when computing the k -th node pool $\mathcal{N}^{(n,k)}$ from the $(k-1)$ -th node pool $\mathcal{N}^{(n,k-1)}$ ($k \geq 1$), the sets $\mathcal{N}_L^{(n,k)}$, $\mathcal{N}_X^{(n,k)}$, and $\mathcal{N}_A^{(n,k)}$ are defined as follows:

$$\begin{aligned} \mathcal{N}_L^{(n,k)} &\stackrel{\text{def}}{=} \left\{ \bigoplus_{a \in T} a : T \subseteq \mathcal{N}^{(n,k-1)}, 1 \leq |T| \leq |\mathcal{N}^{(n,k-1)}| \right\}, \\ \mathcal{N}_X^{(n,k)} &\stackrel{\text{def}}{=} \{a \oplus b : a, b \in \mathcal{N}^{(n,k-1)}, a \leq b\}, \\ \mathcal{N}_A^{(n,k)} &\stackrel{\text{def}}{=} \{a \cdot b : a, b \in \mathcal{N}^{(n,k-1)}, a \leq b\}. \end{aligned}$$

For a 3-bit S-box circuit, consider the following node pool chain (abbr. *node chain*).

$$\mathcal{N}^{(3,0)} \rightarrow \mathcal{N}_L^{(3,1)} \rightarrow \mathcal{N}_A^{(3,2)}.$$

For $i_0 = 0x55, i_1 = 0x33$, and $i_2 = 0x0F$, the chain is constructed as follows:

$$\begin{aligned} \mathcal{N}^{(3,0)} &= \{0, i_0, i_1, i_2\}, \\ \mathcal{N}_L^{(3,1)} &= \{0, i_0, i_1, i_2, i_0 \oplus i_1, i_0 \oplus i_2, i_1 \oplus i_2, i_0 \oplus i_1 \oplus i_2\}, \\ \mathcal{N}_A^{(3,2)} &= \{0, i_0, i_1, i_2, i_0 \oplus i_1, i_0 \oplus i_2, i_1 \oplus i_2, i_0 \oplus i_1 \oplus i_2, \\ &\quad i_0 \oplus i_0 i_1, i_0 \oplus i_0 i_2, \dots, i_0 \oplus i_1 \oplus i_0 i_2 \oplus i_1 i_2\}. \end{aligned}$$

Terminology of SLICE Technique. SLICE consists of two sub-techniques:

- *slicing*: cutting the circuit based on a specific value of $\mathcal{AD}$ (the linear layer between each $\mathcal{AD}$ segment is cut depending on the circuit structure) (cf. Sects. 3.4–3.5)
- *segmenting*: dividing each 2^n -bit element in a given node pool $\mathcal{N}$ into 2^k sub-bitstrings of 2^{n-k} bits, where $1 \leq k < n$ (cf. Sect. 3.5)

3.3 Limiting the Search Space by Node Pool Chain Length

Our first objective is to construct a node pool chain that contains all Boolean functions realizable with AND depth 1. For $k \geq 2$, consider the following chain:

$$\begin{aligned} \mathcal{N}^{(n,0)} &\rightarrow \mathcal{N}_L^{(n,1)} \rightarrow \mathcal{N}_A^{(n,2)}, & \text{for } k = 2, \\ \mathcal{N}^{(n,0)} &\rightarrow \mathcal{N}_L^{(n,1)} \rightarrow \mathcal{N}_A^{(n,2)} \rightarrow \mathcal{N}_X^{(n,3)} \rightarrow \mathcal{N}_X^{(n,4)} \rightarrow \dots \rightarrow \mathcal{N}_X^{(n,k)}, & \text{for } k > 2. \end{aligned}$$

Throughout this paper, the construction of a node pool chain always begins with the initial node pool and follows the structure of the above chain by default.

Since the above chain contains only one AND gate layer, the final node pool includes only functions realizable with AND depth 0 or 1, i.e., at most *quadratic*. Increasing the node-chain length broadens the circuit space but also enlarges the search space, so we must limit the chain length appropriately.

Mirwald and Schnorr [MS92] showed the following property, which establishes the maximum multiplicative complexity (MC) of a quadratic Boolean function.

Property 1 ([MS92]). The MC of an n -bit quadratic Boolean function is at most $\lfloor n/2 \rfloor$.

We obtain the following property.

Property 2. For any $n(\geq 3)$ -bit quadratic Boolean function whose bitsliced representation is s , there exists an $\alpha \in \mathcal{N}_L^{(n,1)}$ such that $s \oplus \alpha \in \mathcal{N}^{(n,2+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$ and $s \in \mathcal{N}^{(n,3+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$.

Proof. Let the MC of s be denoted by MC_s . We decompose s as $s = s_{\text{product}} \oplus \alpha$, where s_{product} is the XOR of MC_s products of linear forms. Then, we have:

$$s_{\text{product}} = s \oplus \alpha \in \begin{cases} \mathcal{N}_X^{(n,2+\lceil \log_2(MC_s) \rceil)}, & \text{for } n > 3, \\ \mathcal{N}_A^{(n,2)}, & \text{for } n = 3. \end{cases}$$

This implies that

$$\begin{cases} s \oplus \alpha \in \mathcal{N}_X^{(n,2+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}, & \text{for } n > 3, \\ s \oplus \alpha \in \mathcal{N}_A^{(n,2)}, & \text{for } n = 3. \end{cases}$$

Since $\alpha \in \mathcal{N}_L^{(n,1)} \subseteq \mathcal{N}_X^{(n,2+\lceil \log_2(MC_s) \rceil)}$, we get $s \in \mathcal{N}_X^{(n,3+\lceil \log_2(MC_s) \rceil)}$. $\square$

Example. The 4-bit Boolean function $f(x) = x_0 \oplus x_1 \oplus x_2 \oplus x_0 x_1 \oplus x_0 x_2 \oplus x_1 x_2 \oplus x_0 x_3 \oplus x_1 x_3$ (whose bitsliced representation is s) has a MC of 1. It can be implemented using a single AND gate in the following three ways:

$$\begin{aligned} f(x) &= (x_0 \oplus x_1)(x_0 \oplus x_2 \oplus x_3) \oplus x_1 \oplus x_2 \\ &= (x_0 \oplus x_1)(x_1 \oplus x_2 \oplus x_3) \oplus x_0 \oplus x_2 \\ &= (x_0 \oplus x_2 \oplus x_3)(x_1 \oplus x_2 \oplus x_3) \oplus x_0 \oplus x_1 \oplus x_3. \end{aligned}$$

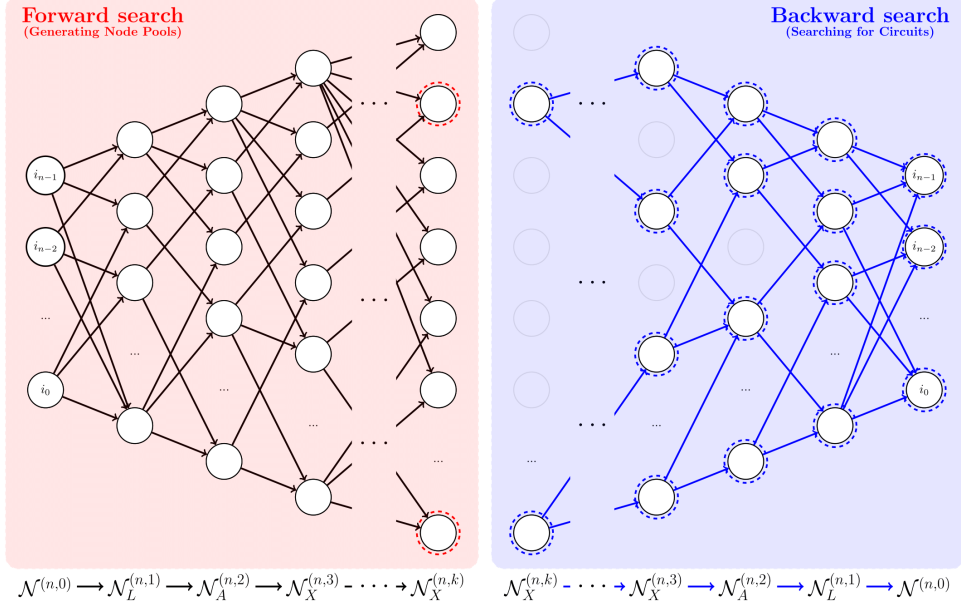


Figure 2: Searching for low-depth circuits via forward-backward techniques.

Each of these implementations includes additional linear terms beyond the product of linear forms. According to Property 2, $s \oplus \alpha \in \mathcal{N}_X^{(4,3)}$ and $s \in \mathcal{N}_X^{(4,4)}$, but it also holds $s \oplus \alpha \in \mathcal{N}_A^{(4,2)}$ and $s \in \mathcal{N}_X^{(4,3)}$. This is because $MC_s < \lfloor n/2 \rfloor$.

This fact implies that for any target s and $n > 3$, there exists an integer j ($1 \leq j \leq \lceil \log_2(\lfloor n/2 \rfloor) \rceil$), nodes $a, b \in \mathcal{N}^{(n,1+j)}$, and $\alpha \in \mathcal{N}_L^{(n,1)}$ such that $a \oplus b = s \oplus \alpha$ or $a \cdot b = s \oplus \alpha$. If $MC_s < \lfloor n/2 \rfloor$, then $j < \lceil \log_2(\lfloor n/2 \rfloor) \rceil$. We call such a pair (a, b) *XOR split* or *AND split* data, which plays an important role in our search. Since deeper node chains generate more circuits, we search in chains up to the point by Property 2 or longer even when $MC_s < \lfloor n/2 \rfloor$.

3.4 Generating Low-Depth S-Box Circuits within AND Depth 1

Each s_i of the S-box $S = (s_{n-1}, \dots, s_0)$ is called a *target* (denoted s_i). For each target, we divide circuit construction into a *forward phase* and a *backward phase*. The forward phase builds a node-pool chain from $\mathcal{N}^{(n,0)}$ to find a representation of the target and records auxiliary data for efficiency. The backward phase then decomposes the target into elements of $\mathcal{N}^{(n,0)}$ using this data. Figure 2 provides an overview of the two-phase construction process.

We repeat the two phases to generate multiple candidates per target, and then select and merge candidates that maximize reuse of AND gate outputs across targets. To illustrate the two-phase process in more detail, we use the MISTY 3-bit S-box as a concrete example throughout both the forward and backward phases.

Forward Phase. The forward phase consists of two main tasks: constructing the node pool chain and generating the dictionary *Dict* for use in the backward phase. It consists of three steps: node generation, *Dict* initialization, and *Dict* update. While the process of node generation follows directly from Definition 1, constructing *Dict* involves some additional steps.

F-Step 1: Node Generation Process. According to Property 2, for each target s_i of the S-box, there exists $\alpha \in \mathcal{N}_L^{(n,1)}$ such that $s_i \oplus \alpha \in \mathcal{N}_X^{(n,2+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$. The node pool chain is constructed only up to $\mathcal{N}^{(n,1+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$, as $s_i \oplus \alpha$ can be represented using XOR/AND operations on elements in the preceding pool. For example, when $n = 3$, the maximum MC of a 3-bit S-box is 1, and constructing the chain $\mathcal{N}^{(3,0)} \rightarrow \mathcal{N}_L^{(3,1)} \rightarrow \mathcal{N}_A^{(3,2)} \rightarrow \mathcal{N}_X^{(3,3)}$ ensures that S-box targets lie in $\mathcal{N}_X^{(3,3)}$. Here, node generation stops at $\mathcal{N}_A^{(3,2)}$, since the backward phase (Algorithm 2) represents s_i using elements from this pool. See lines 6–12 in the MISTY case study below for *F-Step 1*.

F-Step 2: Dict Initialization. Each element of $\mathcal{N}_L^{(n,1)} \setminus \{0\}$ corresponds one-to-one to a subset of $\mathcal{N}^{(n,0)} \setminus \{0\}$, according to Definition 1. For example, when $n = 3$, the element $i_0 \oplus i_1 \oplus i_2 \in \mathcal{N}_L^{(3,1)} \setminus \{0\}$ corresponds to the subset $\{i_0, i_1, i_2\} \subset \mathcal{N}^{(3,0)} \setminus \{0\}$. For each $a \in \mathcal{N}_L^{(n,1)} \setminus \{0\}$, we generate and store a dictionary *Dict* in a key-value format, where the key is a and the value is the corresponding subset of $\mathcal{N}^{(n,0)} \setminus \{0\}$. See lines 14–19 in the MISTY case study below for *F-Step 2*.

F-Step 3: Dict Update. Similarly to *F-Step 2*, each element in $\mathcal{N}_A^{(n,2)}$ corresponds to a one- or two-element subset of $\mathcal{N}_L^{(n,1)}$. By Definition 1, every element $a \cdot b \in \mathcal{N}_A^{(n,2)}$ maps to a subset $\{a, b\} \subset \mathcal{N}_L^{(n,1)}$, and we update *Dict* by setting the key to $a \cdot b$ and the value to $\{Dict[a], Dict[b]\}$. If $a = b$, the key becomes $a \in \mathcal{N}_A^{(n,2)}$ and the value is $\{a\} \subset \mathcal{N}_L^{(n,1)}$, which was already added to *Dict* in *F-Step 2* and is therefore not reconsidered. If needed, a new *Dict* can be built for each element of $\mathcal{N}_X^{(n,k)}$ ($k \geq 3$). The initialization and update steps are given in Algorithm 1. See lines 21–37 in the MISTY case study below for *F-Step 3*.

Algorithm 1 *Dict* construction for backward phase (initialization and update)

Input: $\mathcal{N}^{(n,0)}, \mathcal{N}_L^{(n,1)}$

Output: *Dict*

```

Dict  $\leftarrow$  empty dictionary
for  $T \subseteq \mathcal{N}^{(n,0)}$  where  $1 \leq |T| \leq |\mathcal{N}^{(n,0)}|$  do
    Dict $[\bigoplus_{a \in T} a] \leftarrow T$ 
for  $a \cdot b$  where  $a, b \in \mathcal{N}_L^{(n,1)}, a < b$  do
    Dict $[a \cdot b] \leftarrow (Dict[a], Dict[b])$ 
return Dict

```

Backward Phase. It consists of decomposing each target s_i of the S-box—where there exists $\alpha \in \mathcal{N}_L^{(n,1)}$ such that $s_i \oplus \alpha \in \mathcal{N}_X^{(n,2+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$ —into elements of $\mathcal{N}^{(n,1+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$, and then transforming them into a circuit over $\mathcal{N}^{(n,0)}$ using the dictionary *Dict*. This phase comprises two steps: the XOR split process and the decode to circuit step.

B-Step 1: XOR Split Process. In *F-Step 1*, we construct the node pool chain only up to the layer immediately preceding the nodes that contain the target $s_i \oplus \alpha$ for some $\alpha \in \mathcal{N}_L^{(n,1)}$. Consider the case where the node pool chain $\mathcal{N}^{(3,0)} \rightarrow \mathcal{N}_L^{(3,1)} \rightarrow \mathcal{N}_A^{(3,2)} \rightarrow \mathcal{N}_X^{(3,3)}$ has been computed and stored up to $\mathcal{N}_A^{(3,2)}$. The goal of the XOR split process is to find all pairs (a, b) satisfying $s_i \oplus \alpha = a \oplus b$, where $\alpha \in \mathcal{N}_L^{(3,1)}$, and $a, b \in \mathcal{N}_A^{(3,2)}$. Since $s_i \in \mathcal{N}_X^{(3,3)}$, $\alpha = 0$ in this case. To do so, we first construct the set $T = \{a \oplus s_i \oplus \alpha \mid a \in \mathcal{N}_A^{(3,2)}\}$ and compute the intersection $\mathcal{N}_A^{(3,2)} \cap T$. This allows us to recover all valid pairs (a, b) such that $s_i \oplus \alpha = a \oplus b$. This process is generalized in Algorithm 2.

From Property 2, we know that $s_i \oplus \alpha \in \mathcal{N}_X^{(n,2+\lceil \log_2(\lfloor n/2 \rfloor) \rceil)}$. Therefore, the node pool input to Algorithm 2 is $\mathcal{N}^{(n,k)}$, where $k > 2$. If necessary, the XOR split process can be

applied repeatedly to perform multiple-level decomposition of the target. See lines 39–46 in the MISTY case study below for *B-Step 1*.

Algorithm 2 XOR split process

Input: target s_i , $\mathcal{N}$, α ($= 0$ or bitsliced representation of a linear function)

Output: *ret*

$s_i \leftarrow s_i \oplus \alpha$

$T \leftarrow \{a \oplus s_i \mid a \in \mathcal{N}\}$

$T \leftarrow \mathcal{N} \cap T$

$ret \leftarrow \{sorted(a, a \oplus s_i) \mid a \in T\}$

▷Remove duplicates

return *ret*

B-Step 2: Decode to Circuit. In the previous step, the target s_i was expressed as the XOR of two elements in $\mathcal{N}_A^{(3,2)}$ through the split process, which applies in the same way for any $n \neq 3$.² In this step, each of these split elements is further decomposed into subsets of initial nodes $(i_0, \dots, i_{n-1})$, which are then interpreted using AND and XOR operations to complete the final circuit. Since we have a dictionary *Dict* that stores each element of $\mathcal{N}_A^{(n,2)}$ as a key and maps it to a corresponding subset (or pair of subsets) of $\mathcal{N}^{(n,0)}$, we can express s_i using only subsets of $\mathcal{N}^{(n,0)}$. More specifically, for a key $a \in \mathcal{N}_A^{(n,2)}$, *Dict* returns a value of either the form T_i or (T_j, T_k) , where $T_i, T_j, T_k \subseteq \mathcal{N}^{(n,0)}$. According to the construction rules of *Dict*, if $a \in \mathcal{N}_L^{(n,1)}$, the value is T_i ; otherwise, if $a \in \mathcal{N}_A^{(n,2)} \setminus \mathcal{N}_L^{(n,1)}$, the value is a pair (T_j, T_k) . Thus, based on the number of subsets returned, we can determine whether an AND operation is required. Finally, within each subset, all elements are combined using XOR operations, enabling us to construct the final circuit. See lines 48–73 in the MISTY case study below for *B-Step 2*.

Case Study - MISTY. We now present an example of the above method for constructing a 1AD circuit. The following is the procedure to construct the target s_2 of the 3-bit S-box *S2* from MISTY [CDL15], which we selected as our example. This procedure can be naturally extended to the construction of n -bit S-box circuits.

```

1  [Select Target] Determine the target  $s_i$  to be constructed from the given S-box
2  -  $S2 \leftarrow (2, 5, 6, 4, 0, 1, 3, 7)$ 
3  - Bitsliced representation of  $S2 \leftarrow (0x71, 0xa3, 0x47)$ 
4  -  $s_2 \leftarrow 0x71$ 
5
6  [Forward Step 1] Since all 1AD circuits for the 3-bit S-box can be generated within  $\mathcal{N}_X^{(3,3)}$ , precompute
7  up to  $\mathcal{N}_A^{(3,2)}$ 
8  -  $\mathcal{N}^{(3,0)} \leftarrow \{0, 0x0f, 0x33, 0x55\}$ 
9  -  $\mathcal{N}_L^{(3,1)} \leftarrow \{0, 0x0f, 0x33, 0x3c, 0x55, 0x5a, 0x66, 0x69\}$ 
10 -  $\mathcal{N}_A^{(3,2)} \leftarrow \{0, 0x03, 0x05, 0x06, 0x09, 0x0a, 0x0c, 0x0f, 0x11, 0x12, 0x14, 0x18,$ 
11  $0x21, 0x22, 0x24, 0x28, 0x30, 0x33, 0x3c, 0x41, 0x42, 0x44, 0x48,$ 
12  $0x50, 0x55, 0x5a, 0x60, 0x66, 0x69\}$ 
13
14 [Forward Step 2] Store in Dict each key-value pair  $a : \{b_0, \dots, b_m\}$  ( $m \geq 0$ ), where  $a \in \mathcal{N}_L^{(3,1)} \setminus \{0\}$ 
15 is the key, and the value is a subset  $\{b_0, \dots, b_m\} \subseteq \mathcal{N}^{(3,0)}$  such that  $a = \bigoplus_{b_i \in \{b_0, \dots, b_m\}} b_i$ , that is
16  $Dict[a] = \{b_0, \dots, b_m\}$ 
17 -  $Dict \leftarrow \{0x0f : (\{0x0f\}), \quad 0x33 : (\{0x33\}), \quad 0x55 : (\{0x55\}),$ 
18  $0x3c : (\{0x0f, 0x33\}), \quad 0x5a : (\{0x0f, 0x55\}), \quad 0x66 : (\{0x33, 0x55\}),$ 
19  $0x69 : (\{0x0f, 0x33, 0x55\})\}$ 
20
21 [Forward Step 3] Store in Dict each key-value pair  $a : (Dict[b], Dict[c])$ , where  $a \in \mathcal{N}_A^{(3,2)}$  is the key,
22 and  $b, c \in \mathcal{N}_L^{(3,1)}$  are such that  $a = b \cdot c$ 
-  $Dict \leftarrow \{0x0f : (\{0x0f\}), \quad 0x33 : (\{0x33\}), \quad 0x55 : (\{0x55\}),$ 

```

²If the split process is applied multiple times, the target may be expressed as the XOR of more than two elements in $\mathcal{N}_A^{(n,2)}$.

```

23      0x3c: ({0x0f, 0x33}), 0x5a: ({0x0f, 0x55}), 0x66: ({0x33, 0x55}),
24      0x69: ({0x0f, 0x33, 0x55}),
25      ▷ updated below
26      0x03: ({0x33}, {0x0f}),          0x05: ({0x55}, {0x0f}),
27      0x06: ({0x33, 0x55}, {0x0f}),    0x09: ({0x0f}, {0x0f, 0x33, 0x55}),
28      0x0a: ({0x0f}, {0x0f, 0x55}),    0x0c: ({0x0f}, {0x0f, 0x33}),
29      0x11: ({0x33}, {0x55}),          0x12: ({0x33}, {0x0f, 0x55}),
30      0x14: ({0x55}, {0x0f, 0x33}),    0x18: ({0x0f, 0x33}, {0x0f, 0x55}),
31      0x21: ({0x33}, {0x0f, 0x33, 0x55}), 0x22: ({0x33}, {0x33, 0x55}),
32      0x24: ({0x33, 0x55}, {0x0f, 0x33}), 0x28: ({0x0f, 0x33}, {0x0f, 0x33, 0x55}),
33      0x30: ({0x33}, {0x0f, 0x33}),    0x41: ({0x55}, {0x0f, 0x33, 0x55}),
34      0x42: ({0x33, 0x55}, {0x0f, 0x55}), 0x44: ({0x33, 0x55}, {0x55}),
35      0x48: ({0x0f, 0x33, 0x55}, {0x0f, 0x55}), 0x50: ({0x55}, {0x0f, 0x55}),
36      0x60: ({0x33, 0x55}, {0x0f, 0x33, 0x55}) }
37
38
39 [Backward Step 1] Decompose the selected  $s_2 \in \mathcal{N}_X^{(3,3)}$  into elements of  $\mathcal{N}_A^{(3,2)}$ 
40 ▷ Compute  $T = \{a \oplus s_2 \mid a \in \mathcal{N}_A^{(3,2)}\}$ 
41 -  $T \leftarrow \{0x11, 0x21, 0x30, 0x33, 0x35, 0x39, 0x41, 0x50, 0x53, 0x55, 0x59, 0x60,$ 
42    $0x63, 0x65, 0x69, 0x71, 0x72, 0x74, 0x77, 0x78, 0x7b, 0x7d\}$ 
43 ▷ Decompose  $s_2$  by finding the intersection of  $\mathcal{N}_A^{(3,2)}$  and  $T$ 
44 -  $\mathcal{N}_A^{(3,2)} \cap T = \{0x11, 0x21, 0x30, 0x33, 0x41, 0x50, 0x55, 0x60, 0x69\}$ 
45 -  $Splited \leftarrow \{\{0x11, 0x60\}, \{0x18, 0x69\}, \{0x21, 0x50\},$ 
46    $\{0x24, 0x55\}, \{0x30, 0x41\}, \{0x33, 0x42\}\}$ 
47
48 [Backward Step 2] Construct the final circuit by decoding the decomposition using the precomputed Dict
49 ▷ Update Splited with Dict
50 -  $Splited \leftarrow \{(Dict[0x11], Dict[0x60]), (Dict[0x18], Dict[0x69]), (Dict[0x21], Dict[0x50]),$ 
51    $(Dict[0x24], Dict[0x55]), (Dict[0x30], Dict[0x41]), (Dict[0x33], Dict[0x42])\}$ 
52    $= \{(\{0x33\}, \{0x55\}), (\{0x33, 0x55\}, \{0x0f, 0x33, 0x55\}),$ 
53      $(\{0x0f, 0x33\}, \{0x0f, 0x55\}), (\{0x0f, 0x33, 0x55\}),$ 
54      $(\{0x33\}, \{0x0f, 0x33, 0x55\}), (\{0x55\}, \{0x0f, 0x55\}),$ 
55      $(\{0x33, 0x55\}, \{0x0f, 0x33\}), (\{0x55\}),$ 
56      $(\{0x33\}, \{0x0f, 0x33\}), (\{0x55\}, \{0x0f, 0x33, 0x55\}),$ 
57      $(\{0x33\}), (\{0x33, 0x55\}, \{0x0f, 0x55\})\}$ 
58 ▷ Encode into circuit (1): reconstruct the corresponding expressions
59 -  $circuits \leftarrow \{$ 
60    $(AND(0x33, 0x55) \oplus AND((0x33 \oplus 0x55), (0x0f \oplus 0x33 \oplus 0x55))),$ 
61    $(AND((0x0f \oplus 0x33), (0x0f \oplus 0x55)) \oplus 0x0f \oplus 0x33 \oplus 0x55),$ 
62    $(AND(0x33, (0x0f \oplus 0x33 \oplus 0x55)) \oplus AND(0x55, (0x0f \oplus 0x55))),$ 
63    $(AND((0x33 \oplus 0x55), (0x0f \oplus 0x33)) \oplus 0x55),$ 
64    $(AND(0x33, (0x0f \oplus 0x33)) \oplus AND(0x55, (0x0f \oplus 0x33 \oplus 0x55))),$ 
65    $(AND((0x33 \oplus 0x55), (0x0f \oplus 0x55)) \oplus 0x33)\}$ 
66 ▷ Encode into circuit (2): translate  $x_2 \leftarrow i_2 = 0x0f$ ,  $x_1 \leftarrow i_1 = 0x33$ , and  $x_0 \leftarrow i_0 = 0x55$  (Denote  $x_j$  as
67   the input bit of the S-box, and  $i_j$  as the bitsliced representation of the S-box input)
68 -  $circuits \leftarrow \{$ 
69    $(AND(x_1, x_0) \oplus AND((x_1 \oplus x_0), (x_2 \oplus x_1 \oplus x_0))),$  ▷Depth:4
70    $(AND((x_2 \oplus x_1), (x_2 \oplus x_0)) \oplus x_2 \oplus x_1 \oplus x_0),$  ▷Depth:3
71    $(AND(x_1, (x_2 \oplus x_1 \oplus x_0)) \oplus AND(x_0, (x_2 \oplus x_0))),$  ▷Depth:4
72    $(AND((x_1 \oplus x_0), (x_2 \oplus x_1)) \oplus x_0),$  ▷Depth:3
73    $(AND(x_1, (x_2 \oplus x_1)) \oplus AND(x_0, (x_2 \oplus x_1 \oplus x_0))),$  ▷Depth:4
74    $(AND((x_1 \oplus x_0), (x_2 \oplus x_0)) \oplus x_1)\}$  ▷Depth:3

```

In this case study, six distinct circuits are generated for s_2 , with depths ranging from 3 to 4. After constructing the circuits for s_0 and s_1 as well, the maximum depth among the circuits for s_0 , s_1 , and s_2 can be regarded as the circuit depth of the S-box. Since only one of the three output bits is constructed in this example, the entire S-box implementation is discussed in Sect. 3.6.

Observations on NOT Operation. Although NOT gates are inherently low-cost, when combined with AND, OR, or XOR gates they are transformed into more efficient NAND, NOR, or XNOR gates, and such combinations may also reduce logical depth. In particular, when a NOT gate appears on the input of an AND operation, it can be eliminated or pushed outward, thus enlarging the node pool and enabling the exploration of more diverse circuits. The following illustrates examples of such circuit diversification:

$$\begin{aligned}
 AND(NOT(b), c) &= AND(b, c) \oplus c, \\
 AND(NOT(b), NOT(c)) &= NOT(AND(b, c) \oplus b \oplus c)
 \end{aligned}$$

Accordingly, we hereafter adopt the following extended node pool.

Definition 2 (Extended Node Pool). The extended initial node pool $\tilde{\mathcal{N}}^{(n,0)}$ is defined by adding $\mathbb{1}$ to $\mathcal{N}^{(n,0)}$, with $\mathbb{1}$ denoting the NOT operation:

$$\tilde{\mathcal{N}}^{(n,0)} = \{0, \mathbb{1}, i_0, i_1, \dots, i_{n-1}\}.$$

In the above definition, $\mathbb{1}$ denotes $\underbrace{11 \dots 1}_{2^n \text{ bits of } 1}$, such that $XOR(a, \mathbb{1}) = \neg a$ and $AND(a, \mathbb{1}) = a$.

a. Based on this extended initial node pool, the node pool chain can also be constructed as in the original case. For example, when $n = 3$, the chain can be constructed as $\tilde{\mathcal{N}}^{(3,0)} \rightarrow \tilde{\mathcal{N}}_L^{(3,1)} \rightarrow \tilde{\mathcal{N}}_A^{(3,2)}$. Due to the effect of the NOT operation, we have $|\tilde{\mathcal{N}}_L^{(3,1)}| > |\mathcal{N}_L^{(3,1)}|$ and $|\tilde{\mathcal{N}}_A^{(3,2)}| > |\mathcal{N}_A^{(3,2)}|$, which naturally increases the number of circuit candidates to be considered.

Remark 1. In the *Dict* construction, the mapping between subsets of $\tilde{\mathcal{N}}^{(n,0)}$ and elements of $\tilde{\mathcal{N}}_L^{(n,1)}$ remains one-to-one. However, since an element of $\tilde{\mathcal{N}}_A^{(n,2)} \setminus \{0\}$ corresponds to three pairs in $\tilde{\mathcal{N}}_L^{(n,1)}$, we can search much more circuit candidates.

Remark 2. For clarity, we do not include NOT gates in the MISTY example; if they are included, the number of circuits for s_2 with $\mathcal{D} = 3$ increases from 3 to 7 and with $\mathcal{D} = 4$ from 3 to 44. Although the depth remains unchanged in the MISTY example, the increased number of circuits provides a broader search space, potentially improving the depth metric. In fact, NOT gates are used throughout our study, and all subsequent descriptions assume the inclusion of NOT gates.

3.5 Segmenting Technique: Breaking the AND Depth 1 Barrier

In this section, we introduce the *segmenting* technique, which extends the search space by decomposing elements of $\tilde{\mathcal{N}}$. Each element in $\tilde{\mathcal{N}}^{(n,k)}$ consists of 2^n bits. The *segmenting* technique aims to efficiently explore circuits by observing only 2^{n-l} bits out of the full 2^n -bit space where $1 \leq l < n$. Using this technique, we extend the circuit construction method to handle circuits with larger bit-sizes or with AND depth 2. While the explanation here focuses on an 8-bit example, the method can be naturally extended to general n -bit cases. Figure 3 illustrates an overview of our proposed *segmenting* technique in terms of a node pool chain.

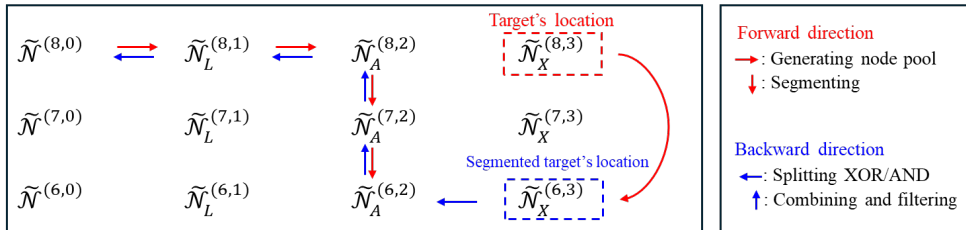


Figure 3: Circuit construction via segmenting the 8-bit node pool into 6 bits.

Forward Phase with Segmenting. The forward phase with *segmenting* is almost identical to the one described in Sect. 3.4. The only difference is in *F-Step 1*, where node generation is performed over the reduced bit-width from segmentation. The following steps proceed as before.

FS-Step 1: Expanded Node Generation Process. The original algorithm generated nodes only up to the memory limit. We overcome this with the *segmenting* technique, which allows circuit construction beyond direct enumeration.

Consider the adopted bitsliced representation. An important observation is that the circuit construction process is entirely carried out using bitwise operations. All computations occur within each bit position (i.e., column-wise), and there are no operations performed across rows. Hence, we can observe the following property:

Property 3. The set obtained by splitting each 2^n -bit element of $\tilde{\mathcal{N}}^{(n,k)}$ into 2^l subbitstrings of length 2^{n-l} bits is identical to $\tilde{\mathcal{N}}^{(n-l,k)}$ after excluding duplication, where $1 \leq l < n$.

We refer to *segmenting with l -bit compression* as the case described in Property 3. Segmenting with l -bit compression is sometimes abbreviated to l -bit compression.

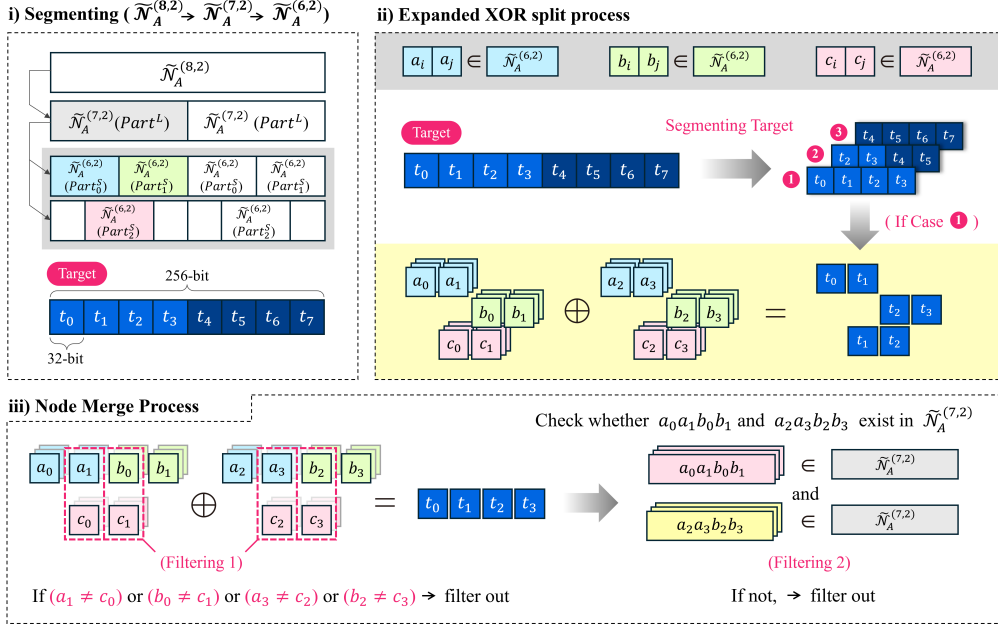
The bitsliced representations of identity permutations for 3–8 bit inputs are shown below. The top portion presents color-coded representations of 3–7 bit widths. Below, the merged 3–8 bit representation is shown, with the 8-bit case further segmented into four parts under 2-bit compression.

Orange: 3-bit, Blue: 4-bit, Teal: 5-bit, Violet: 6-bit, Red: 7-bit, Black: 8-bit									
[n-bit bitsliced representation]									
i_6								0000000000000000	FFFFFFFFFFFFFFFF
i_5							00000000	FFFFFFFF	00000000FFFFFFFF
i_4						0000FFFF	0000FFFF	0000FFFF	0000FFFF
i_3					00FF	00FF	00FF	00FF	00FF
i_2	0F	0F	0F	0F	0F	0F	0F	0F	0F
i_1	33	3333	33333333	3333333333333333	33333333333333333333333333333333	33333333333333333333333333333333	33333333333333333333333333333333	33333333333333333333333333333333	33333333333333333333333333333333
i_0	55	5555	55555555	5555555555555555	55555555555555555555555555555555	55555555555555555555555555555555	55555555555555555555555555555555	55555555555555555555555555555555	55555555555555555555555555555555
1	FF	FFFF	FFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
	3-bit	4-bit	5-bit	6-bit	7-bit	8-bit	8-bit	8-bit	8-bit
[3–8-bit merged bitsliced representation, segmented into 4 parts] (2-bit compression)									
i_7	0000000000000000	0000000000000000	0000000000000000	0000000000000000	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
i_6	0000000000000000	FFFFFFFFFFFFFFFF	0000000000000000	0000000000000000	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
i_5	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF	00000000FFFFFFFF
i_4	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF	0000FFFF0000FFFF
i_3	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF	00FF00FF00FF00FF
i_2	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F	0F0F0F0F0F0F0F
i_1	3333333333333333	3333333333333333	3333333333333333	3333333333333333	3333333333333333	3333333333333333	3333333333333333	3333333333333333	3333333333333333
i_0	5555555555555555	5555555555555555	5555555555555555	5555555555555555	5555555555555555	5555555555555555	5555555555555555	5555555555555555	5555555555555555
1	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF	FFFFFFFFFFFFFFFF
	part1	part2	part3	part4					

If we apply 1-bit compression to $\tilde{\mathcal{N}}^{(8,0)} = \{0, 1, i_0^8, i_1^8, \dots, i_7^8\}$, we can observe that both the left and right halves correspond to $\tilde{\mathcal{N}}^{(7,0)} = \{0, 1, i_0^7, i_1^7, \dots, i_6^7\}$ after excluding duplication. Furthermore, applying 2-bit compression to $\tilde{\mathcal{N}}^{(8,0)}$ results in four parts, each of which is identical to $\tilde{\mathcal{N}}^{(6,0)}$. This demonstrates that applying l -bit compression to a node pool $\tilde{\mathcal{N}}^{(n,*)}$ effectively switches the original node chain to another node chain of the form $\tilde{\mathcal{N}}^{(n-l,*)}$. We utilize this property to extend the feasible length of the node pool chain in the forward phase. For instance, if storing all elements of $\tilde{\mathcal{N}}^{(n,k)}$ from $\tilde{\mathcal{N}}_X^{(n,k-1)}$ is infeasible, we apply *segmenting* to convert $\tilde{\mathcal{N}}_X^{(n,k-1)}$ into $\tilde{\mathcal{N}}_X^{(n-1,k-1)}$ and then expand it to $\tilde{\mathcal{N}}_X^{(n-1,k)}$. If further extension remains infeasible, we apply another *segmenting* step to transform it into $\tilde{\mathcal{N}}_X^{(n-2,k-1)}$ and attempt to extend the chain again.

FS-Step 2.3: Dict Initialization and Update. When using *segmenting*, the initialization and update processes of *Dict* follow the same procedures as the forward phase described in Sect. 3.4.

Backward Phase with Segmenting. The backward phase with *segmenting* is largely similar to the backward phase described in Sect. 3.4. However, due to the change in node


 Figure 4: Overview of the node merging process when $\text{target} \in \tilde{\mathcal{N}}_X^{(8,3)}$.

size, *B-Step 1*—the XOR split process—must be performed repeatedly. Aside from this, the remaining steps follow the same procedure.

BS-Step 1: Expanded XOR Split Process. When *segmenting* is applied, the *XOR split process* is carried out in two distinct steps. The first step performs the standard separation of values, as in the original process. The second step involves merging the *segmented* parts into a larger node. This merging step is performed only when feasible. Each step is described as follows.

BS-Step 1-1: XOR Split Process. *BS-Step 1-1* involves expressing a value in $\tilde{\mathcal{N}}_X^{(n,k)}$ as a combination of elements from $\tilde{\mathcal{N}}_X^{(n,k-1)}$. This process follows the same procedure as the previous *XOR split process*. However, it is important to note that the nodes have been constructed through *segmenting*.

Suppose the *target* belongs to $\tilde{\mathcal{N}}_X^{(n,k)}$, where applying the *XOR split process* requires nodes up to $\tilde{\mathcal{N}}_X^{(n,k-1)}$. When full node construction is infeasible, we use *segmenting* to store $\tilde{\mathcal{N}}_X^{(n-l,k-1)}$. In this case, the *target* must be segmented, and the *XOR split process* applied to each part. For clarity, we refer to each part obtained through *segmenting* the *target* as a *segment* of the *target*.

BS-Step 1-2: Node Merge Process. Here we address a critical issue: since circuits are constructed by partially observing the *target* through *segmenting*, there is no guarantee that the resulting circuit is correct when the backward phase aims to reach $\tilde{\mathcal{N}}^{(n-l,0)}$. Among several possible resolutions, we enhance reliability by performing a node merging process, which restores *segmenting*-based nodes to their original form. For instance, after *segmenting* and expressing a value using elements from $\tilde{\mathcal{N}}^{(n-l,k-1)}$ via the *XOR split process*, we re-express it with elements from $\tilde{\mathcal{N}}^{(n,k-1)}$. This merging process allows us to filter out invalid circuits. Figure 4 provides an overview of our node merging procedure.

Before presenting the explanation, we define the auxiliary terms $Part^L$ and $Part^S$ to aid understanding. $Part^L$ denotes a larger node, while $Part^S$ refers to a smaller node obtained by dividing $Part^L$ into two halves, denoted $Part_0^S$ and $Part_1^S$. Additionally, we define $Part_2^S$ as a *segmenting*-based portion of $Part^L$ that, rather than being created by bisecting, overlaps partly with $Part_0^S$ and partly with $Part_1^S$. Although $Part_2^S$ contains exactly the same data as $Part_0^S$ and $Part_1^S$, we use a separate notation depending on the location for descriptive convenience.

Figure 4 illustrates a process where $\tilde{N}^{(8,2)}$ is 1-bit compressed to obtain $\tilde{N}^{(7,2)}$, which is further 1-bit compressed to yield $\tilde{N}^{(6,2)}$. From the perspective of $\tilde{N}^{(8,2)}$, $\tilde{N}^{(7,2)}$ acts as a $Part^S$, whereas from the perspective of $\tilde{N}^{(6,2)}$, it acts as a $Part^L$. Similarly, $\tilde{N}^{(6,2)}$ serves as a $Part^S$ relative to $\tilde{N}^{(7,2)}$. This example illustrates the merging process of $\tilde{N}^{(6,2)}$ back into $\tilde{N}^{(7,2)}$, which can be repeated to eventually reconstruct $\tilde{N}^{(8,2)}$.

Suppose the *XOR split process* has been applied to segmented target t_0, t_1, t_2 , and t_3 , which are split according to the positions of colored $Part_0^S, Part_1^S$, and $Part_2^S$, respectively. Then, we can obtain three types of expressions.

$$\begin{aligned} a_0||a_1 \oplus a_2||a_3 &= t_0||t_1 \text{ where } a_0||a_1, a_2||a_3 \in Part_0^S, t_0||t_1 \in \text{Segmented Target} \\ b_0||b_1 \oplus b_2||b_3 &= t_2||t_3 \text{ where } b_0||b_1, b_2||b_3 \in Part_1^S, t_2||t_3 \in \text{Segmented Target} \\ c_0||c_1 \oplus c_2||c_3 &= t_1||t_2 \text{ where } c_0||c_1, c_2||c_3 \in Part_2^S, t_1||t_2 \in \text{Segmented Target} \end{aligned}$$

Let us now consider the positional relationships of the $Part^S$ elements. To obtain a correct circuit, the following conditions must hold: $a_1 = c_0$, $a_3 = c_2$, $b_0 = c_1$, and $b_2 = c_3$. If even one of these conditions is not satisfied, the resulting circuit is invalid. This process can be interpreted as checking whether the pairs (a_1, b_0) and (a_3, b_2) are contained in the pink-colored set of pairs, including (c_0, c_1) and (c_2, c_3) . It is important to remember that the circuit is considered in the context of the bitsliced representation. This step performs the first round of filtering. By repeating this process, we can more efficiently construct $Part_0^S || Part_1^S$.

If the concatenated value is indeed a realizable one, it must exist in $Part^L$. At this point, since we initially held two values through the *XOR split*, both merged values must exist in $Part^L$ and collectively be able to reconstruct a segment of the *target*. This enables the second round of filtering, through which a significant number of invalid circuits are discarded. In this filtering step, we also check whether $a_0a_1b_0b_1, a_2a_3b_2b_3 \in \tilde{N}^{(7,2)}$. By this process, an XOR expression over $\tilde{N}^{(6,2)}$ can be lifted to $\tilde{N}^{(7,2)}$, and by repeating, ultimately to $\tilde{N}^{(8,2)}$.

This merging procedure guarantees correct exhaustive search under the prescribed AND depth bound. By Property 3, every element of $\tilde{N}^{(n-l,k)}$ corresponds to a segment of some element in $\tilde{N}^{(n,k)}$, so enumerating all candidates at reduced bit-width and lifting them via merging is equivalent to exhaustively checking all realizable circuits at the original bit-width. The two-stage filtering checks boundary consistency and then tests membership in higher-level node pools, and it is sound and complete, removing invalid or unrealizable candidates while preserving all valid ones. As a result, exhaustive search remains correct even when directly enumerating $\tilde{N}^{(n,k)}$ is infeasible due to memory constraints.

BS-Step 2: Decode to Circuit. This step can be performed in the same manner as described in Sect. 3.4. Table 2 presents the range of circuit exploration determined by whether the *segmenting* technique was required, as confirmed through our experiments. The target range corresponds to the node pool chain used in Sect. 4. Entries marked with $\circ$ indicate cases where *segmenting* was not required, while $\bullet$ indicates that it was necessary. For example, since node pools up to $\tilde{N}_A^{(12,2)}$ can be generated, $\tilde{N}_X^{(12,3)}$ is explored without segmenting and marked with $\circ$. In contrast, $\tilde{N}_X^{(12,4)}$ cannot be generated directly due to memory limits, requiring *segmenting* and is marked with $\bullet$. We used an AMD Ryzen 9

Table 2: Investigable ranges for node pools with and without segmenting.

$\mathcal{AD}$	-	3-bit	4-bit	5-bit	6-bit	7-bit	8-bit	10-bit	12-bit	16-bit
0	$\tilde{\mathcal{N}}_L^{(n,1)}$	o	o	o	o	o	o	o	o	o
	$\tilde{\mathcal{N}}_A^{(n,2)}$	o	o	o	o	o	o	o	o	o
1	$\tilde{\mathcal{N}}_X^{(n,3)}$	o	o	o	o	o	o	o	o	o
	$\tilde{\mathcal{N}}_X^{(n,4)}$	o	o	o	o	o	o	•	•	•
2	$\tilde{\mathcal{N}}_A^{(n,5)}$	o	o	o	o	o	•	-	-	-
	$\tilde{\mathcal{N}}_X^{(n,6)}$	o	o	o	•	•	•	-	-	-

o: Without segmenting; •: With segmenting; -: Not investigated

7950X (16 cores @ 4.5 GHz), 64 GB RAM, and a Gigabyte X670 AORUS ELITE AX mainboard for our experiments.

2.4D Segmenting. For clarity, we explained our technique assuming segmenting is applied at the level of $1\mathcal{AD}$, although it can be naturally extended to $2\mathcal{AD}$. In the $2\mathcal{AD}$ setting, we adopt by default a node pool chain where $\tilde{\mathcal{N}}_X$ and $\tilde{\mathcal{N}}_A$ alternate in succession, with $j = \lceil \log_2(\lfloor n/2 \rfloor) \rceil$.

$$\tilde{\mathcal{N}}^{(n,0)} \rightarrow \tilde{\mathcal{N}}_L^{(n,1)} \rightarrow \tilde{\mathcal{N}}_A^{(n,2)} \rightarrow \tilde{\mathcal{N}}_X^{(n,3)} \rightarrow \dots \rightarrow \tilde{\mathcal{N}}_X^{(n,j+3)} \rightarrow \tilde{\mathcal{N}}_A^{(n,j+4)} \rightarrow \tilde{\mathcal{N}}_X^{(n,j+5)} \rightarrow \dots$$

We heuristically observed that the default chain is sufficient to cover the target circuit, with construction beyond $\tilde{\mathcal{N}}_A^{(n,j+4)}$ extended solely by $\tilde{\mathcal{N}}_X$. With this node pool chain, the segmenting technique for $1\mathcal{AD}$ circuits extends naturally to $2\mathcal{AD}$ circuits, and is used to construct the AES S-box circuits in Sect. 4.

3.6 Completing the Subcircuit

One important point not yet stated is that all circuit constructions so far have n -bit inputs and a single-bit output. In practice, to construct a circuit with an m -bit output, the SLICE technique must be applied m times, once for each output bit. However, it is reasonable to question whether combining m independently constructed output circuits into a single S-box is feasible in practice.

Let us consider the MISTY S-box example again. Previously, we generated the circuits for only the 1-bit output s_2 ; we now repeat the process to also construct s_1 and s_0 . Our experiments showed that the numbers of circuits for s_1 and s_0 match that of s_2 , so including NOT gates, the total number of realizable MISTY S-box circuits is 51^3 (see Remark 2). By extending the node pool by one additional layer, the number of combinable MISTY S-box circuits increases to 348873^3 , which is considerably larger. For an 8-bit S-box, over 2^{24} circuits may be generated for each 1-bit output, making exhaustive exploration of all combinations infeasible. Hence, we first address how to complete each subcircuit.

We address this problem with two complementary strategies: minimizing circuit depth and minimizing $\#\mathcal{NL}$, applicable in any order. For an n -bit input and m -bit output circuit, the overall depth equals the maximum depth of its m outputs. Thus, we measure the minimum depth for each of the m output branches individually, identify the optimal achievable depth for the circuit, and then filter circuit combinations based on this depth threshold. For the extended node chain for the MISTY S-box, the total number of combinable circuits is 348873^3 , but if we limit the depth to 3, the number is reduced to 4358^3 .

The second strategy is to prioritize a small number of nonlinear gates ($\#\mathcal{NL}$). For circuits of the same depth, we favor low $\#\mathcal{NL}$ while keeping a few extra candidates to explore better ADP trade-offs, where $\#\mathcal{L}$ may also be used as a secondary criterion. In the MISTY S-box example, our construction process applies an XOR split, through which each target is expressed using elements from $\mathcal{N}_A^{(n,2)}$. When different m -bit outputs are represented by the same element in $\mathcal{N}_A^{(n,2)}$ that is not in $\mathcal{N}_L^{(n,1)}$, this indicates a shared AND gate. This property can be leveraged to minimize the overall number of nonlinear

gates $\#\mathcal{NL}$ used in the combined circuit. To this end, we adopt a greedy algorithm to select output combinations that minimize $\#\mathcal{NL}$. For instance, given a 4-bit output, we first evaluate all pairs such as (s_0, s_1) , (s_0, s_2) , (s_0, s_3) , (s_1, s_2) , (s_1, s_3) , and (s_2, s_3) , and retain only those with the smallest $\#\mathcal{NL}$. Subsequently, we combine these pairs to evaluate combinations such as $((s_0, s_1), (s_2, s_3))$, $((s_0, s_2), (s_1, s_3))$, and $((s_0, s_3), (s_1, s_2))$, and again filter based on the minimum $\#\mathcal{NL}$.

3.7 Further Reducing Circuit Depth by Combining Subcircuits

The SLICE technique first *slices* a circuit into subcircuits and then optimizes each independently, with boundaries chosen heuristically under the limits of the *segmenting* technique. For example, if it is feasible to explore up to $\tilde{\mathcal{N}}_X^{(n,6)}$, then each subcircuit can be optimized accordingly. These optimized subcircuits typically follow the structural pattern of *linear layer* (L) – *nonlinear layer* (A) – *linear layer* (X). See Definition 1 for the notation of L , A , and X .

Now, consider concatenating two such subcircuits, yielding the overall structure $(L \rightarrow A \rightarrow X) \rightarrow (L \rightarrow A \rightarrow X)$. Note that the two consecutive linear layers ($X \rightarrow L$) can potentially be merged into a single linear layer. Whether such a merge of the $X \rightarrow L$ connection is beneficial can be determined through exhaustive search. As explained in Sect. 3.6, the number of subcircuits can be significantly reduced, so exhaustive search is computationally feasible. This process may lead to a new configuration with an even smaller overall depth.

3.8 Applicability and Versatility of SLICE

This section highlights three features of SLICE that are most relevant to its application and performance.

1. **Semi-automated workflow with a path to full automation.** At present, SLICE is semi-automated: once a baseline circuit has been partitioned, the optimization of sliced subcircuits and their recombination are performed automatically, but the initial $\mathcal{AD}$ -based slicing of the baseline circuit still involves heuristic choices and some expert input. A concrete path toward full automation is to compute the $\mathcal{AD}$ of each wire, automatically generate multiple candidate slice boundaries around the $1\mathcal{AD}$ level, extend to $2\mathcal{AD}$ when necessary, and then evaluate the candidates by running SLICE and ranking the results by depth first and then cost (e.g., nonlinear and linear gate counts). Thus, the current manual effort is largely limited to selecting or validating the slicing strategy, and future work could reduce even this step to a small set of generic rules or learned circuit-pattern features, making the overall workflow close to fully automated at the cost of additional computation.
2. **Broad applicability and parallel output construction.** SLICE applies to any S-box. The node-pool chain is independent of the S-box, so once precomputed, adapting SLICE to other S-boxes requires relatively low effort. After the chain is built, each output bit is constructed in parallel during the backward phase, enabling simultaneous generation of multiple outputs.
3. **Truth-table-to-circuit workflow.** Although SLICE starts from a circuit description, it can also be applied when only the truth table is available, even for new S-boxes with unknown design principles. In this case, we first derive the coordinate functions from the table (e.g., in ANF form), synthesize a baseline bitsliced circuit, and then apply the same SLICE pipeline for refinement. For small S-boxes such as **Ascon** and the cube x^3 , we find that baseline circuits synthesized from the truth table already yield competitive results, indicating limited sensitivity to initialization.

4 Applications to AES S-Box Circuits and Others

In this section, we demonstrate the effectiveness of the SLICE technique by applying it to S-box circuits of AES to reduce their depth. For brevity, we present only this case; a similar workflow applies to smaller S-boxes such as **Ascon**, **Dillon**, and cube x^3 , which are typically easier to explore and often do not require segmenting. The bit-level circuits of all new designs are provided in Appendix C.

We focus on two types of AES S-box circuits: those proposed by Hadžić and Bloem, and by Canright. Currently known Hadžić and Bloem’s circuit is $\mathcal{AD}=3$, $\mathcal{D}=14$, $\#\mathcal{NL}=42$, and Canright’s circuit is $\mathcal{AD}=4$ at $\mathcal{D}=16$, $\#\mathcal{NL}=33$ and $\mathcal{AD}=5$ at $\mathcal{D}=17$, $\#\mathcal{NL}=32$. Our objective is to find circuits that are more optimized than the current state-of-the-art (SOTA). Figure 5 shows the results by *slicing* heuristically for two types of circuits. For Hadžić and Bloem’s circuit, we perform two types of *slicing* to derive new insights. Table 3 summarizes the results along with the node chains employed in our slicing process.

We first create one AES circuit by combining Areas A, B, and C. If each Area is viewed independently, $\mathcal{D}=14$. Since the end of Area A and the start of Area B are both linear, there is room for depth reduction if the circuits are combined appropriately. In our experiments, we found $\mathcal{D}=8$ cases by exhaustively combining all cases of Areas A and B. Building on these intermediate results, we exhaustively combined Area C and discovered a circuit with $\mathcal{D}=12$.

We then turn to another construction that combines Areas D, E, and F. In our experiments, the best combination of the three areas resulted in a circuit with $\mathcal{D}=12$. The key point is that the optimized Area D has $\mathcal{D}=7$, and with a small number of additional XOR operations it can be extended to cover up to Area B, achieving $\mathcal{D}=8$. This matches the depth obtained by optimally combining the *sliced* subcircuits A and B. Although we did not obtain a more optimal depth, this result demonstrates that *segmenting* successfully

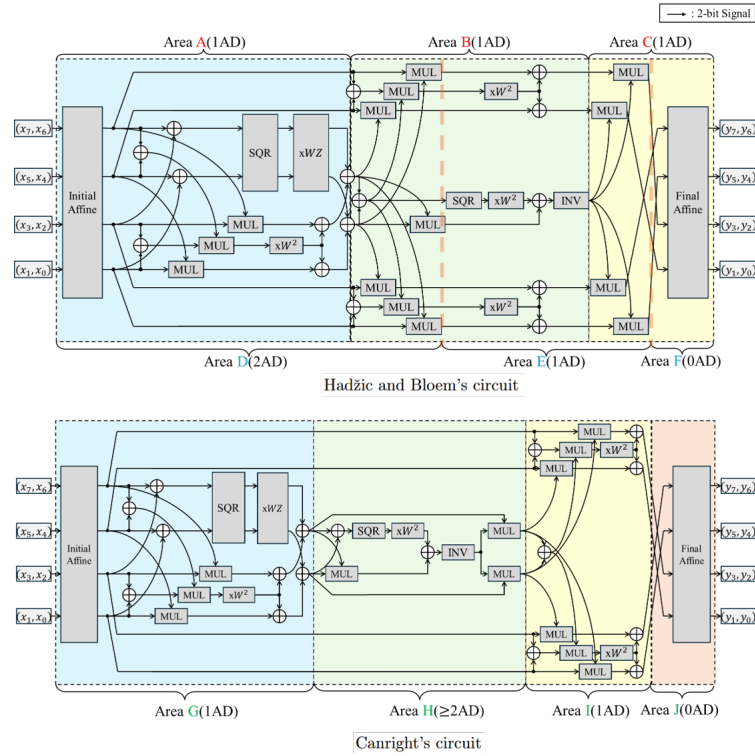


Figure 5: The *sliced* circuits of Hadžić and Bloem (top) and Canright (bottom).

Table 3: Summary of results of applying SLICE technique to each *sliced* area.

Area	$\mathcal{D}$	$\mathcal{AD}$	$\#\mathcal{NL}$	Forward Node chain
A	5	1	9	$\tilde{\mathcal{N}}^{(8,0)} \rightarrow \tilde{\mathcal{N}}_L^{(8,1)} \rightarrow \tilde{\mathcal{N}}_A^{(8,2)} \rightarrow \tilde{\mathcal{N}}_X^{(8,3)} \rightarrow \tilde{\mathcal{N}}_X^{(8,4)}$
B	4	1	21	$\tilde{\mathcal{N}}^{(12,0)} \rightarrow \tilde{\mathcal{N}}_L^{(12,1)} \rightarrow \tilde{\mathcal{N}}_A^{(12,2)} \rightarrow \tilde{\mathcal{N}}_X^{(8,3)} \rightarrow \tilde{\mathcal{N}}_X^{(8,4)}$
C	5	1	12	$\tilde{\mathcal{N}}^{(10,0)} \rightarrow \tilde{\mathcal{N}}_L^{(10,1)} \rightarrow \tilde{\mathcal{N}}_A^{(10,2)} \rightarrow \tilde{\mathcal{N}}_X^{(8,3)} \rightarrow \tilde{\mathcal{N}}_X^{(8,4)}$
Combined(A, B, C)	12 or 13	3	42	-
D	8	2	30	$\tilde{\mathcal{N}}^{(8,0)} \rightarrow \tilde{\mathcal{N}}_L^{(8,1)} \rightarrow \tilde{\mathcal{N}}_A^{(8,2)} \rightarrow \tilde{\mathcal{N}}_X^{(5,3)} \rightarrow \tilde{\mathcal{N}}_X^{(5,4)} \rightarrow \tilde{\mathcal{N}}_A^{(5,5)} \rightarrow \tilde{\mathcal{N}}_X^{(5,6)} \rightarrow \tilde{\mathcal{N}}_X^{(5,7)}$
E	3	1	12	$\tilde{\mathcal{N}}^{(16,0)} \rightarrow \tilde{\mathcal{N}}_L^{(16,1)} \rightarrow \tilde{\mathcal{N}}_A^{(16,2)} \rightarrow \tilde{\mathcal{N}}_X^{(16,3)}$
F	3	0	0	$\mathcal{N}^{(8,0)} \rightarrow \mathcal{N}_L^{(8,1)}$
Combined(D, E, F)	12 or 13	3	42	-
G	5	1	9	$\tilde{\mathcal{N}}^{(8,0)} \rightarrow \tilde{\mathcal{N}}_L^{(8,1)} \rightarrow \tilde{\mathcal{N}}_A^{(8,2)} \rightarrow \tilde{\mathcal{N}}_X^{(8,3)} \rightarrow \tilde{\mathcal{N}}_X^{(8,4)}$
H	4	2	6	$\tilde{\mathcal{N}}^{(4,0)} \rightarrow \tilde{\mathcal{N}}_L^{(4,1)} \rightarrow \tilde{\mathcal{N}}_A^{(4,2)} \rightarrow \tilde{\mathcal{N}}_X^{(4,3)} \rightarrow \tilde{\mathcal{N}}_X^{(4,4)} \rightarrow \tilde{\mathcal{N}}_A^{(4,5)} \rightarrow \tilde{\mathcal{N}}_X^{(4,6)}$
I	5	1	18	$\tilde{\mathcal{N}}^{(12,0)} \rightarrow \tilde{\mathcal{N}}_L^{(12,1)} \rightarrow \tilde{\mathcal{N}}_A^{(12,2)} \rightarrow \tilde{\mathcal{N}}_X^{(8,3)} \rightarrow \tilde{\mathcal{N}}_X^{(8,4)}$
J	3	0	0	$\mathcal{N}^{(8,0)} \rightarrow \mathcal{N}_L^{(8,1)}$
Combined(G, H, I, J)	14	4	33	-

*Red: Segmenting is used
*Blue: Node where *target* exists

breaks the AND depth 1 barrier.

As a final case, we construct an AES circuit by combining Areas G , H , I , and J . Area G is the same as Area A , and Area H can be viewed as a 4-bit S-box. While various optimization tools exist for 4-bit S-boxes, our focus is to identify an Area H circuit that can be efficiently integrated with Areas G and I in terms of depth. We successfully implemented Area H with a circuit having $\mathcal{AD} = 2$ and $\mathcal{D} = 4$, and confirmed that extending $\mathcal{AD}$ to 3 increases $\mathcal{D}$ but decreases $\#\mathcal{NL}$. Areas I and J can be investigated separately, but their combination reveals that the *target* belongs to $\tilde{\mathcal{N}}_X^{(12,4)}$. We investigated efficient integrations of areas, first combining G and H and then I and J . Combining G and H yielded a subcircuit with $\mathcal{D} = 9$ or $\mathcal{D} = 11$ depending on the $\mathcal{AD}$ of H , while combining I and J yielded a subcircuit with $\mathcal{D} = 6$. By combining these two subcircuits, we obtained a circuit with $\mathcal{D} = 14$, $\mathcal{AD} = 4$, and $\#\mathcal{NL} = 33$, achieving a lower $\mathcal{D}$ than existing designs.

The search for the 12- and 13-depth AES S-box circuits required about 10 hours in total on the experimental platform described in Sect. 3.5, including roughly 8 hours for low-depth subcircuit search and 2 hours for circuit combination. Peak memory usage during the overall process was 48 GB. For smaller S-boxes, the computational cost was significantly lower in our experiments, as the Dillion 6-bit case took about 3 hours, whereas the Ascon and cube x^3 cases finished within a few minutes. In general, $1\mathcal{AD}$ -based partitioning leads to smaller subcircuits with lower time and memory costs, whereas $2\mathcal{AD}$ -based partitioning can enable stronger optimization at the expense of a larger search space.

While SLICE supports effective optimization within each sliced subcircuit, it does not explicitly account for interactions across different subcircuits. In practice, such cross-subcircuit interactions can reduce overall depth, suggesting that there may also be room to further reduce linear gates such as XORs across slices. We therefore additionally adopt eBPD (see Sect. 5), which can optimize XOR gates while taking nonlinear dependencies into account by merging linear layers across slices.

5 Further Optimization for Compact S-Box Circuits and Bit-Level Masked Implementations

In this section, we refine the heuristic search framework of [JBKK25] to reduce the XOR gate count in some of our proposed S-box circuits while preserving low depth, thereby improving circuit compactness without sacrificing latency. We then customize the Compress tool [CGM⁺24] to our setting to derive masked bit-level S-box circuits for our proposed designs and to evaluate their side-channel resistance.

before masking; post-masking logic optimization then absorbs the extra NOTs where possible, so this preprocessing does not impact area or latency compared to using explicit masked gadgets. The results are presented in Table 4.

6 Conclusions

In this paper, we proposed **SLICE**, a novel technique for constructing low-depth S-box circuits, which comprises two sub-techniques: *slicing* and *segmenting*. Leveraging this technique, we discovered several new depth records: for the **AES** S-box, we found a 13-depth circuit with improved gate count and further pushed the depth down to 12, yielding the first 12-depth circuit; for the **Ascon** S-box, we obtained the first 4-depth circuit. Although the **AES** S-box still has room for further optimization, the **Ascon** S-box appears to have reached the optimal depth under the current structure. Beyond these two ciphers, **SLICE** also yields improved low-depth circuits for **Dillon** and the cube x^3 S-boxes. Furthermore, we evaluated the resulting circuits using the ADP metric, and showed that our designs achieve the best performance among existing implementations. Notably, for all target S-boxes, our proposed circuits consistently improve ADP over all previously reported implementations. Finally, we assessed the cost of our circuits under bit-level masking, showing that they remain comparable even in side-channel-resistant settings. We expect **SLICE** to facilitate systematic exploration of low-depth, hardware-efficient S-box circuits and to accelerate the discovery of improved implementations for future cryptographic primitives.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2025-00564126) and by the Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (No. RS-2025-25411243).

References

- [BBK⁺13] Begül Bilgin, Andrey Bogdanov, Miroslav Knezevic, Florian Mendel, and Qingju Wang. Fides: Lightweight authenticated cipher with side-channel resistance for constrained hardware. In Guido Bertoni and Jean-Sébastien Coron, editors, *CHES 2013*, volume 8086 of *Lecture Notes in Computer Science*, pages 142–158. Springer, 2013.
- [BDM25] Marie Bolzer, Sébastien Duval, and Marine Minier. A new tool to find lightweight (and, xor) implementations of quadratic vectorial boolean functions up to dimension 9. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.
- [BDMD⁺20] Begül Bilgin, Lauren De Meyer, Sébastien Duval, Itamar Levi, and François-Xavier Standaert. Low and depth and efficient inverses: a guide on s-boxes for low-latency masking. *IACR Transactions on Symmetric Cryptology*, 2020(1):144–184, May 2020.
- [BDMW10] Keith A Browning, John F Dillon, MT McQuistan, and Alan J Wolfe. An apn permutation in dimension six. *Finite Fields: theory and applications*, 518:33–42, 2010.

- [BFP19] Joan Boyar, Magnus Gausdal Find, and René Peralta. Small low-depth circuits for cryptographic applications. *Cryptogr. Commun.*, 11(1):109–127, 2019.
- [BMP08] Joan Boyar, Philip Matthews, and René Peralta. On the shortest linear straight-line program for computing linear forms. In *Mathematical Foundations of Computer Science 2008, MFCS 2008*, volume 5162 of *Lecture Notes in Computer Science*, pages 168–179. Springer, 2008.
- [BP10] Joan Boyar and René Peralta. A new combinational logic minimization technique with applications to cryptology. In *Experimental Algorithms, 9th International Symposium, SEA 2010*, volume 6049 of *Lecture Notes in Computer Science*, pages 178–189. Springer, 2010.
- [BP12] Joan Boyar and René Peralta. A small depth-16 circuit for the AES s-box. In *Information Security and Privacy Research - 27th IFIP TC 11 Information Security and Privacy Conference, SEC 2012*, volume 376 of *IFIP Advances in Information and Communication Technology*, pages 287–298. Springer, 2012.
- [BPP⁺17] Subhadeep Banik, Sumit Kumar Pandey, Thomas Peyrin, Yu Sasaki, Siang Meng Sim, and Yosuke Todo. GIFT: A small present - towards reaching the limit of lightweight encryption. In *Cryptographic Hardware and Embedded Systems - CHES 2017*, volume 10529 of *Lecture Notes in Computer Science*, pages 321–345. Springer, 2017.
- [Can05] David Canright. A very compact s-box for AES. In *Cryptographic Hardware and Embedded Systems - CHES 2005*, volume 3659 of *Lecture Notes in Computer Science*, pages 441–455. Springer, 2005.
- [CDL15] Anne Canteaut, Sébastien Duval, and Gaëtan Leurent. Construction of lightweight s-boxes using feistel and MISTY structures. In *Selected Areas in Cryptography - SAC 2015*, volume 9566 of *Lecture Notes in Computer Science*, pages 373–393. Springer, 2015.
- [CDL⁺20] Anne Canteaut, Sébastien Duval, Gaëtan Leurent, María Naya-Plasencia, Léo Perrin, Thomas Pornin, and André Schrottenloher. Saturnin: a suite of lightweight symmetric algorithms for post-quantum security. *IACR Transactions on Symmetric Cryptology*, 2020(S1):160–207, 2020.
- [CGM⁺24] Gaëtan Cassiers, Barbara Gigerl, Stefan Mangard, Charles Momin, and Rishub Nagpal. Compress: Generate small and fast masked pipelined circuits. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):500–529, 2024.
- [CMT16] CMT. Circuit minimization work, 2016. last accessed on: 1st October 2025.
- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.
- [DR02] Joan Daemen and Vincent Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Information Security and Cryptography. Springer, 2002.
- [For06] 3GPP Task Force. Specification of the 3GPP Confidentiality and Integrity Algorithms UEA2 & UIA2. Document 2: SNOW 3G Specification. Technical Report Version 1.1, 3GPP, 2006.

- [HB25] Vedad Hadžić and Roderick Bloem. Efficient and composable masked AES s-box designs using optimized inverters. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):656–683, 2025.
- [IT88] Toshiya Itoh and Shigeo Tsujii. A fast algorithm for computing multiplicative inverses in $\text{gf}(2^m)$ using normal bases. *Information and computation*, 78(3):171–177, 1988.
- [JBK24] Yongjin Jeon, Seungjun Baek, and Jongsung Kim. Towards finding s-box circuits with optimal multiplicative complexity. *IEEE Transactions on Computers*, 73(8):2036–2050, 2024.
- [JBKK25] Yongjin Jeon, Seungjun Baek, Giyoon Kim, and Jongsung Kim. A framework for generating s-box circuits with boyar-peralta algorithm-based heuristics, and its applications to aes, snow3g, and saturnin. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(1):586–631, 2025.
- [JPST17] Jérémy Jean, Thomas Peyrin, Siang Meng Sim, and Jade Tourteaux. Optimizing implementations of lightweight building blocks. *IACR Transactions on Symmetric Cryptology*, 2017(4):130–168, 2017.
- [LSL⁺19] Shun Li, Siwei Sun, Chaoyun Li, Zihao Wei, and Lei Hu. Constructing low-latency involutory MDS matrices with lightweight circuits. *IACR Transactions on Symmetric Cryptology*, 2019(1):84–117, 2019.
- [LXZZ21] Da Lin, Zejun Xiang, Xiangyong Zeng, and Shasha Zhang. A framework to optimize implementations of matrices. In *Topics in Cryptology - CT-RSA 2021 - Cryptographers' Track at the RSA Conference 2021*, volume 12704 of *Lecture Notes in Computer Science*, pages 609–632. Springer, 2021.
- [LZW23] Qun Liu, Zheng Zhao, and Meiqin Wang. Improved heuristics for low-latency implementations of linear layers. In *Topics in Cryptology - CT-RSA 2023 - Cryptographers' Track at the RSA Conference 2023*, volume 13871 of *Lecture Notes in Computer Science*, pages 524–550. Springer, 2023.
- [MS92] Roland Mirwald and Claus-Peter Schnorr. The multiplicative complexity of quadratic boolean forms. *Theoretical Computer Science*, 102(2):307–328, 1992.
- [PUB16] Léo Perrin, Aleksei Udovenko, and Alex Biryukov. Cryptanalysis of a theorem: Decomposing the only known solution to the big APN problem. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016*, volume 9815 of *Lecture Notes in Computer Science*, pages 93–122. Springer, 2016.
- [RTA18] Arash Reyhani-Masoleh, Mostafa M. I. Taha, and Doaa Ashmawy. Smashing the implementation records of AES s-box. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(2):298–336, 2018.
- [SFX23] Haotian Shi, Xiutao Feng, and Shengyuan Xu. A framework with improved heuristics to optimize low-latency implementations of linear layers. *IACR Transactions on Symmetric Cryptology*, 2023(4):489–510, 2023.
- [STMC⁺25] Meltem Sönmez Turan, Kerry McKay, Donghoon Chang, Jinkeon Kang, and John Kelsey. Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions. Technical report, National Institute of Standards and Technology, 2025.

- [Sto16] Ko Stoffelen. Optimizing s-box implementations for several criteria using SAT solvers. In *Fast Software Encryption - 23rd International Conference, FSE 2016*, volume 9783 of *Lecture Notes in Computer Science*, pages 140–160. Springer, 2016.
- [TP20] Quan Quan Tan and Thomas Peyrin. Improved heuristics for short linear programs. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(1):203–230, 2020.
- [VSP18] Andrea Visconti, Chiara Valentina Schiavo, and René Peralta. Improved upper bounds for the expected circuit complexity of dense systems of linear equations over $GF(2)$. *Information processing letters*, 137:1–5, 2018.
- [XZL⁺20] Zejun Xiang, Xiangyong Zeng, Da Lin, Zhenzhen Bao, and Shasha Zhang. Optimizing implementations of linear layers. *IACR Transactions on Symmetric Cryptology*, 2020(2):120–145, 2020.

A Target S-Box Circuits

This appendix summarizes the target S-box circuits considered in this work, including those of AES, Ascon, Dillon, and cube x^3 .

A.1 AES S-box Circuits

We briefly review the two AES S-box circuits considered in this work: Canright’s circuit and the circuit of Hadžić and Bloem. Both implement the standard AES S-box, consisting of a multiplicative inversion in $GF(2^8)$ followed by an affine transformation over $GF(2)$, but differ in how the inversion layer is decomposed and implemented.

The AES S-box operates on an 8-bit input $\mathbf{x}$ in two steps.

1. (*Multiplicative Inverse*) Let $\mathbf{t} = \mathbf{x}^{-1}$, the multiplicative inverse in $GF(2^8)$.
2. (*Affine Transformation*) Compute $\mathbf{y} = B\mathbf{t} \oplus \mathbf{b}$, with the constant bit matrix B and byte $\mathbf{b}$ expressed as:

$$\begin{pmatrix} y_7 \\ y_6 \\ y_5 \\ y_4 \\ y_3 \\ y_2 \\ y_1 \\ y_0 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} t_7 \\ t_6 \\ t_5 \\ t_4 \\ t_3 \\ t_2 \\ t_1 \\ t_0 \end{pmatrix} \oplus \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

where y_7 and t_7 are the most significant bits, and all operations are over $GF(2)$.

Canright’s AES S-Box Circuit [Can05]. Following [Can05, BP12], we represent $GF(2^8)$ as a tower field:

- $GF(2^2)$ by adjoining a root W of $x^2 + x + 1$ over $GF(2)$,
- $GF(2^4)$ by adjoining a root Z of $x^2 + x + W^2$ over $GF(2^2)$,
- $GF(2^8)$ by adjoining a root Y of $x^2 + x + WZ$ over $GF(2^4)$.

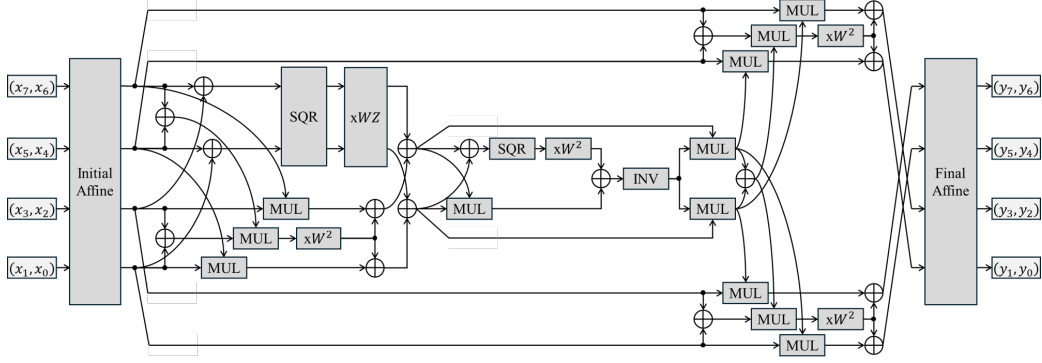


Figure 6: Canright's AES S-box circuit.

We use the bases (W, W^2) for $GF(2^2)$, (Z, Z^4) for $GF(2^4)$, and (Y, Y^{16}) for $GF(2^8)$. Let $A = a_0Y + a_1Y^{16}$ be an arbitrary element in $GF(2^8)$. Using the Itoh-Tsujii approach [IT88], the inverse A^{-1} can be written as:

$$\begin{aligned}
 A^{-1} &= (AA^{16})^{-1}A^{16} \\
 &= ((a_0Y + a_1Y^{16})(a_1Y + a_0Y^{16}))^{-1}(a_1Y + a_0Y^{16}) \\
 &= ((a_0^2 + a_1^2)Y + a_0a_1(Y^2 + Y^{32}))^{-1}(a_1Y + a_0Y^{16}) \\
 &= ((a_0 + a_1)^2Y^{17} + a_0a_1(Y + Y^{16})^2)^{-1}(a_1Y + a_0Y^{16}) \\
 &= ((a_0 + a_1)^2WZ + a_0a_1)^{-1}(a_1Y + a_0Y^{16}). \tag{1}
 \end{aligned}$$

Defining the following intermediate variables,

$$\begin{aligned}
 T_1 &= (a_0 + a_1), \quad T_2 = (WZ)T_1^2, \quad T_3 = a_0a_1, \quad T_4 = T_2 + T_3, \\
 T_5 &= T_4^{-1}, \quad T_6 = T_5a_1, \quad T_7 = T_5a_0,
 \end{aligned}$$

we obtain $A^{-1} = T_6Y + T_7Y^{16}$.

Figure 6 provides a structural overview of this computation. Here, $a_0 = (x_3, x_2, x_1, x_0)$ and $a_1 = (x_7, x_6, x_5, x_4)$, while $T_6 = (y_3, y_2, y_1, y_0)$ and $T_7 = (y_7, y_6, y_5, y_4)$.

Hadžić and Bloem's AES S-Box Circuit [HB25]. Hadžić and Bloem start from the same tower-field decomposition as Canright, but restructure the computation involving T_4^{-1} so as to reduce the overall AND depth. In particular, instead of treating the inversion and the subsequent multiplications separately, they derive a form of $T_4^{-1}a$ that enables more parallel computation.

Let us consider computing $T_4^{-1}a$. Expressing T_4 and a relative to the basis over $GF(2^2)$, we obtain:

$$T_4 = t_1W^2 + t_0W, \quad a = \alpha_1W^2 + \alpha_0W,$$

where t_1, t_0, α_1 , and α_0 are elements of $GF(2^2)$.

T_4^{-1} can be expanded similarly to Equation 1, yielding

$$T_4^{-1} = \underbrace{(t_0t_1 + t_0^2 + t_1^2)^{-1}}_{\rho}(t_1W + t_0W^2).$$

In the expansion, we leverage the relationship $W^2 + W + 1 = 0$ to derive the following expression:

$$\begin{aligned}
 W^3 &= W^2 + W, \\
 W^4 &= W(W^2 + W) = W^3 + W^2 = W.
 \end{aligned}$$

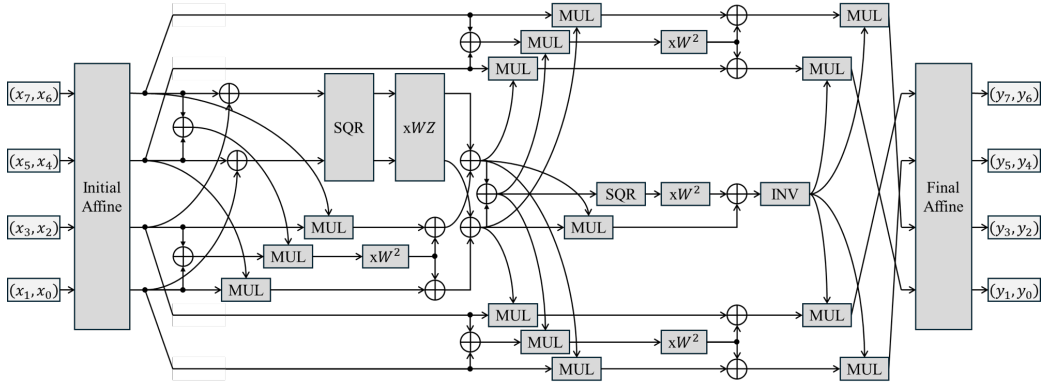


Figure 7: Hadžić and Bloem's AES S-box circuit.

$T_4^{-1}a$ is expressed as:

$$\begin{aligned}
 T_4^{-1}a &= \rho(t_1W + t_0W^2)(\alpha_1W^2 + \alpha_0W) \\
 &= \rho(\alpha_1t_0W^4 + (\alpha_1t_1 + \alpha_0t_0)W^3 + \alpha_0t_1W^2) \\
 &= \rho(\alpha_1t_0W + (\alpha_1t_1 + \alpha_0t_0)(W^2 + W) + \alpha_0t_1W^2) \\
 &= \rho(\alpha_1t_0W^2 + \alpha_0t_1W + \underbrace{(\alpha_1 + \alpha_0)(t_1 + t_0)}_{\theta}(W^2 + W)).
 \end{aligned}$$

Using this result to compute $T_6 = T_4^{-1}a_1$ and $T_7 = T_4^{-1}a_0$, ρ and θ can be processed in parallel. Moreover, given that ρ depends solely on T_4 , it does not need to be computed twice. Figure 7 provides a structural overview of this computation.

A.2 Ascon S-Box Circuit [DEMS21]

Figure 8 illustrates Dobraunig et al.'s Ascon S-box. The circuit implementing the Ascon S-box comprises 5 AND, 11 XOR, and 6 NOT gates. The circuit has depth 5 and AND depth 1. To the best of our knowledge, no prior work has successfully reduced the Ascon S-box's depth.

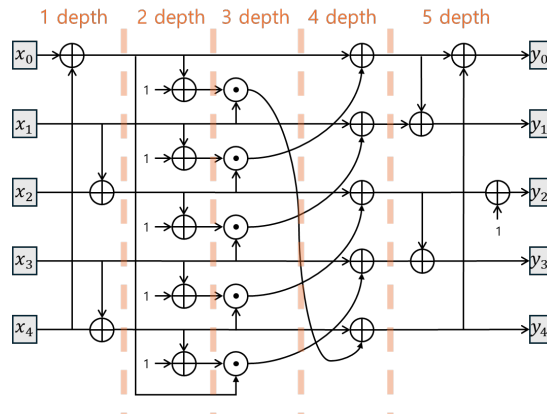


Figure 8: Dobraunig et al.'s Ascon S-box circuit.

A.3 Dillon [PUB16] and Cube x^3 S-box Circuits [BDMD⁺20, BDM25]

Figure 9 illustrates the affine-equivalent core S_I used in the decomposition of the 6-bit Dillon permutation.

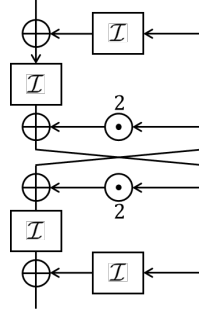


Figure 9: The affine-equivalent core S_I for the Dillon S-box.

More precisely, the permutation is represented in [PUB16] as

$$S_0(x) = B(S_I(A(x) \oplus 9)) \oplus 4,$$

where Figure 9 corresponds to the computation of S_I . The input is split into two 3-bit branches, ℓ and r , viewed as elements of $GF(2^3)$ over the polynomial basis defined by $X^3 + X + 1$. Each I block denotes inversion in $GF(2^3)$, each $\odot 2$ block denotes multiplication by 2 in $GF(2^3)$, and each $\oplus$ denotes branch-wise XOR. The authors in [PUB16] describe how to construct the Dillon S-box, but we were unable to reproduce an explicit circuit instance directly from the description. Instead, we derived a reproducible baseline instance from the proposed structure with minor refinements. The resulting circuit has $\mathcal{D} = 14$, $\mathcal{AD} = 3$, $\#\mathcal{NL} = 12$, $\#\mathcal{L} = 57$, and $\#\mathcal{G} = 69$, as reported in Table 5.

Table 5: Reconstructed Dillon S-box circuit ($\mathcal{D}$: 14, $\mathcal{AD}$: 3, $\#\mathcal{NL}$: 12, $\#\mathcal{L}$: 57, $\#\mathcal{G}$: 69)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_0 \oplus x_1$	3	$g_2 = \text{AND}(x_3, t_{11})$	8	$t_{32} = t_{26} \oplus t_{27}$	10	$g_{10} = \text{AND}(t_{36}, t_{31})$
1	$t_1 = x_3 \oplus x_5$	4	$t_{17} = g_1 \oplus t_{12}$	8	$t_{33} = t_{26} \oplus t_{27}$	10	$g_{11} = \text{AND}(t_{29}, t_{36})$
1	$t_2 = x_0 \oplus x_1$	4	$t_{18} = g_2 \oplus t_{13}$	8	$t_{34} = t_{26} \oplus t_{27}$	11	$t_{47} = g_7 \oplus t_{42}$
1	$t_3 = x_2 \oplus x_3$	4	$t_{19} = t_{14} \oplus t_{15}$	8	$t_{35} = t_{26} \oplus t_{27}$	11	$t_{48} = t_{29} \oplus g_{10}$
1	$t_4 = x_3 \oplus x_5$	5	$t_{20} = t_{17} \oplus t_{18}$	8	$g_6 = \text{AND}(t_{26}, t_{27})$	11	$t_{49} = g_{11} \oplus t_{43}$
1	$t_5 = x_3 \oplus x_4$	5	$t_{21} = t_{17} \oplus t_{18}$	9	$t_{36} = t_{28} \oplus t_{30}$	12	$t_{50} = t_{44} \oplus t_{47}$
1	$t_6 = x_0 \oplus 1$	5	$g_3 = \text{AND}(t_{17}, t_{19})$	9	$t_{37} = t_{29} \oplus t_{31}$	12	$t_{51} = t_{45} \oplus t_{48}$
2	$t_7 = t_0 \oplus t_1$	5	$g_4 = \text{AND}(t_{18}, t_{19})$	9	$t_{38} = t_{28} \oplus g_6$	12	$t_{52} = t_{46} \oplus t_{49}$
2	$t_8 = t_2 \oplus t_3$	5	$g_5 = \text{AND}(t_{17}, t_{18})$	9	$t_{39} = t_{26} \oplus t_{28}$	13	$t_{53} = t_{39} \oplus t_{51}$
2	$t_9 = x_3 \oplus t_4$	6	$t_{22} = g_3 \oplus t_{20}$	9	$t_{40} = t_{28} \oplus t_{35}$	13	$t_{54} = t_{41} \oplus t_{50}$
2	$t_{10} = x_3 \oplus t_6$	6	$t_{23} = t_{17} \oplus g_4$	9	$t_{41} = t_{26} \oplus t_{28}$	14	$y_0 = t_{52} \oplus t_{53}$
2	$t_{11} = t_5 \oplus 1$	6	$t_{24} = t_{19} \oplus g_5$	9	$g_7 = \text{AND}(t_{29}, t_{31})$	8	$y_1 = t_{28}$
2	$g_0 = \text{AND}(t_4, x_3)$	7	$t_{25} = t_{21} \oplus t_{24}$	9	$g_8 = \text{AND}(t_{26}, t_{28})$	13	$y_2 = t_{34} \oplus t_{52}$
3	$t_{12} = t_7 \oplus t_9$	7	$t_{26} = t_{22} \oplus x_3$	9	$g_9 = \text{AND}(t_{27}, t_{28})$	10	$y_3 = t_{40} \oplus 1$
3	$t_{13} = t_4 \oplus t_8$	7	$t_{27} = t_{16} \oplus t_{23}$	10	$t_{42} = t_{29} \oplus t_{36}$	13	$y_4 = t_{52} \oplus t_{27}$
3	$t_{14} = t_4 \oplus t_{11}$	8	$t_{28} = t_{25} \oplus t_4$	10	$t_{43} = t_{36} \oplus t_{37}$	14	$y_5 = t_{52} \oplus t_{54}$
3	$t_{15} = g_0 \oplus t_{10}$	8	$t_{29} = t_4 \oplus t_{27}$	10	$t_{44} = t_{32} \oplus g_8$		
3	$t_{16} = t_4 \oplus t_{11}$	8	$t_{30} = x_3 \oplus t_{26}$	10	$t_{45} = t_{26} \oplus g_9$		
3	$g_1 = \text{AND}(t_4, t_{11})$	8	$t_{31} = t_{11} \oplus t_{26}$	10	$t_{46} = t_{33} \oplus t_{38}$		

For the cube x^3 S-box, [BDMD⁺20] presented explicit 6-bit and 7-bit gate-level circuits, whereas [BDM25] further improved the reported linear and nonlinear gate counts without providing full circuit descriptions, so the circuit depth cannot be determined directly from that work. Since the mapping $x \mapsto x^3$ is quadratic, both the 6-bit and 7-bit cases naturally admit $1\mathcal{AD}$ implementations. Accordingly, besides using the circuits in [BDMD⁺20] as references, baseline implementations can also be synthesized directly from the truth table; in our experiments, such truth-table-derived baselines already yielded competitive results for the cube x^3 S-box.

B Circuit Optimizing Framework Using eBPD Heuristic

Here, we provide an overview of the framework for circuit optimization proposed in [JBKK25], including additional explanations of our refinements.

B.1 Summary of Framework and eBPD Heuristic

The BP heuristic [BP10], originally proposed by Boyar and Peralta to optimize the AES S-box linear layer, was later improved by Tan and Peyrin [TP20] with the RNBP heuristic to avoid biased random selection, and by Li et al. [LSL⁺19] with the BP with depth limit to reduce latency; further details can be found in the respective papers. These two variants were later extended to nonlinear operations, resulting in eRNBP and eBPD [JBKK25], which take circuits as input rather than truth tables. To further reduce input dependence, [JBKK25] also proposed a circuit optimization framework, illustrated in Figure 10, which consists of three steps.

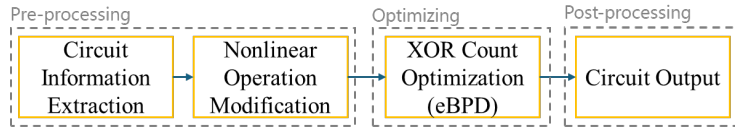


Figure 10: Framework utilizing eBPD heuristic.

B.1.1 Pre-Processing Step.

The pre-processing step is divided into two parts: extracting circuit information and applying transformations to nonlinear gates. The extracted circuit information includes the XOR relations, as illustrated in Table 6 with an example from the GIFT [BPP⁺17] S-box. In this representation, XOR gates are merged into a single linear combination, while the input structures of each nonlinear operator are recorded. NOT gates are discarded at this stage and later reintegrated during the post-processing step to complete the circuit.

Table 6: Extraction of circuit information from GIFT S-box circuit.

Original circuit		Extraction of circuit information and nonlinear gates			
No.	Gate	No.	Gate	No.	Gate
1	$x_1 = x_1 \oplus \text{AND}(x_0, x_2)$	1	$r_0 = x_0$	9	$g_2 = \text{OR}(r_4, r_5)$
2	$t = x_0 \oplus \text{AND}(x_1, x_3)$	2	$r_1 = x_2$	10	$r_6 = x_0 \oplus g_1$
3	$x_2 = x_2 \oplus \text{OR}(t, x_1)$	3	$g_0 = \text{AND}(r_0, r_1)$	11	$r_7 = x_1 \oplus x_2 \oplus x_3 \oplus g_0 \oplus g_2$
4	$x_0 = x_3 \oplus x_2$	4	$r_2 = x_1 \oplus g_0$	12	$g_3 = \text{AND}(r_6, r_7)$
5	$x_1 = x_1 \oplus x_0$	5	$r_3 = x_3$	13	$y_0 = x_2 \oplus x_3 \oplus g_2$
6	$x_0 = x_0 \oplus 1$	6	$g_1 = \text{AND}(r_2, r_3)$	14	$y_1 = x_1 \oplus x_2 \oplus x_3 \oplus g_0 \oplus g_2$
7	$x_2 = x_2 \oplus \text{AND}(t, x_1)$	7	$r_4 = x_0 \oplus g_1$	15	$y_2 = x_2 \oplus g_2 \oplus g_3$
8	$x_3 = t$	8	$r_5 = x_1 \oplus g_0$	16	$y_3 = x_0 \oplus g_1$
		Nodes where NOT gate is used : y_0			

Transformations of nonlinear gates diversify the circuit by constructing equivalent variants, such as adding XOR or NOT gates or changing the gate type, thereby enabling potential performance gains. For large and complex S-box circuits such as the AES S-box, it is infeasible to explore all nonlinear-gate transformations simultaneously. Thus, a greedy mechanism is employed, where each iteration selects the transformation yielding the largest XOR reduction.

B.1.2 Optimizing Step.

The optimization step executes the extended BP-based heuristic, namely eBPD, which unlike BP with depth limit considers not only the outputs of the input linear layer but also

the inputs of nonlinear operations as targets. For example, Table 6 shows $r_0, \dots, r_7$ and $y_0, \dots, y_3$ regarded as targets. When scoring newly constructed XOR gates, the algorithm uses the *distance* between the base and targets, defined as the minimum number of XOR gates required to construct the target within the given depth limit. If a target cannot be expressed solely as the XOR of elements in the base, its distance is defined as ∞ . *Dist* denotes the list enumerating the distance values for all targets.

The eBPD heuristic employs two types of special strategies. The first, called the *pre-emptive strategy*, is triggered when a target has distance 1, in which case it is immediately added to the base. The second, called the *new pre-emptive strategy*, applies when both inputs of a nonlinear operation are in the base but the output is not, in which case the output is immediately added. The pre-emptive strategy accelerates the search for XOR gates, whereas the new pre-emptive strategy enables eBPD to handle nonlinear operations.

B.1.3 Post-Processing Step.

The post-processing step restores the NOT gates discarded during pre-processing to complete the circuit, and among the candidates generated by the eBPD heuristic, it selects the best-performing circuit.

B.2 Refinements in Pre-Processing Step

We revised the purpose of nonlinear operation transformations in the pre-processing step. The first priority of the greedy mechanism to depth is set to obtain a low-depth circuit, as presented in Table 7.

Table 7: Priorities of nonlinear gate transformations in pre-processing.

Measure	Original priority	New priority
Number of XOR gates	First	Second
Depth	-	First

The original framework transforms the input/output nodes of nonlinear gates by transforming AND, OR, NAND, and NOR gates into each other, or by employing the XOR result of two inputs. For example, for two nodes x and y , there exists a transformation where $AND(x, y) = OR(x, y) + x + y$ or $AND(x, y) = AND(x + y, y) + y$. However, the presence of NOT gates in outputs of nonlinear gates, such as $AND(x, y) = NAND(x, y) + 1$, does not affect the XOR information passed to the optimizing step; hence, it does not impact the resulting circuit. This results in the original framework being considered to cover only six transformations of one nonlinear gate. We additionally leverage the fact that adding a NOT gate to the input of a nonlinear gate affects the output. A single AND gate exhibits the following transformations:

$$\begin{aligned}
AND(x, y) &= AND(x + y, y) + y &= AND(x, x + y) + x \\
&= AND(x + 1, y) + y &= AND(x, y + 1) + x &= OR(x, y) + x + y \\
&= AND(x + y + 1, y) &= AND(x + y, y + 1) + x &= OR(x + y, y) + x + y \\
&= AND(x + 1, x + y) + y &= AND(x, x + y + 1) &= OR(x, x + y) + x + y.
\end{aligned}$$

These transformations, relative to XOR information (ignoring NOT operations), are expressed accordingly. Here, x, y present two input nodes, and g denotes the output node of the nonlinear gate before transformation.

$$\begin{aligned}
(x, y) &\rightarrow g, & (x, y) &\rightarrow g + x, & (x, y) &\rightarrow g + y, & (x, y) &\rightarrow g + x + y, \\
(x + y, y) &\rightarrow g, & (x + y, y) &\rightarrow g + x, & (x + y, y) &\rightarrow g + y, & (x + y, y) &\rightarrow g + x + y, \\
(x, x + y) &\rightarrow g, & (x, x + y) &\rightarrow g + x, & (x, x + y) &\rightarrow g + y, & (x, x + y) &\rightarrow g + x + y.
\end{aligned}$$

The OR gate also has the same number of similar transformations, and NAND and NOR gates are not considered. Thus, each nonlinear gate has 12 transformations, and a circuit with k such gates has 12^k .

We also increase the number of nonlinear gates considering transformations in a single iteration of the greedy mechanism. In an iteration, we approach global optimization by considering the transformations of two or more nonlinear gates. Table 8 summarizes these alterations, where b represents the number of nonlinear gates transformations considered at one iteration.

Table 8: Comparison of nonlinear-gate transformation pools in pre-processing.

Pool of investigation subjects	Original	New
All possible transformations of the circuit	6^k	12^k
Transformations considered in one iteration of greedy mechanism	6	12^b

B.3 Redundant Gates Limitation of eBPD Heuristic

The eBPD algorithm has a limitation of generating different XOR gates with the same output value, which requires a solution. To address this limitation, we investigated its root cause. Accordingly, we inferred that the difference in the implementation order occurs when the eBPD algorithm implements nodes with different depths but the same output value. More details will be provided below. In addition, we introduce a technique to reimplement the circuit by reconfiguring it, which is the idea of [LZW23], to improve the performance of eBPD.

In order to determine the cause of the duplicate XOR output value generation, we analyze the process of eBPD. The target node set of eBPD includes not only the output nodes of the circuit but also the input nodes of the nonlinear gates. It sets the depth limit for each target based on the depth limit H input by the user, and then stores it in H_Y . Theorem 1 is used to set the depth limit.

Theorem 1. [LSL⁺19] Let $S = \{v_0, v_1, \dots, v_{n-1}\}$ be a set of nodes, where each v_i has a depth d_i for $i = 0, 1, \dots, n-1$. Then the lower bound of the depth of the circuit implementing $z = v_0 \oplus \dots \oplus v_{n-1}$ is $\lceil \log_2 \sum_{i=0}^{n-1} 2^{d_i} \rceil$.

Table 9 presents how H_Y is set for a 5-/2-bit S-box.

Table 9: XOR information of example with depth limit $H = 4$.

No.	Gate	Initial distance	Minimum depth	Depth limit (H_Y)
1	$r_0 = x_0 \oplus x_1 \oplus x_3 \oplus x_4$	3	2	3
2	$r_1 = x_0 \oplus x_1$	1	1	3
3	$g_0 = \text{AND}(r_0, r_1)$	-	3	4
4	$r_2 = x_0 \oplus x_1 \oplus x_3 \oplus x_4$	3	2	2
5	$r_3 = x_2$	0	0	2
6	$g_1 = \text{AND}(r_2, r_3)$	-	3	3
7	$r_4 = g_1$	∞	3	3
8	$r_5 = x_0 \oplus x_1 \oplus x_3$	2	2	3
9	$g_2 = \text{AND}(r_4, r_5)$	-	4	4
10	$y_0 = g_0$	∞	3	4
11	$y_1 = g_2$	∞	4	4

Here, the target nodes are $r_0, r_1, r_2, r_3, r_4, r_5, y_0, y_1$. Theorem 1 is adopted to calculate the *minimum depth* row, and it can be deduced that the minimum depth of the circuit is 4. Given that $H = 4$, the depth limit of each target is set as in Table 9. The initial Dist is $[3, 1, 3, 0, \infty, 2, \infty, \infty]$ and H_Y is $[3, 3, 2, 2, 3, 3, 4, 4]$. In these settings, eBPD works as in Table 10.

Table 10: eBPD for an example.

No.	Operation	Depth	New <i>Dist</i>	Working principle
1	$t_0 = x_0 \oplus x_1 // r_1$	1	$[2, 0, 2, 0, \infty, 1, \infty, \infty]$	Pre-emptive strategy
2	$t_1 = t_0 \oplus x_3 // r_5$	2	$[1, 0, 2, 0, \infty, 0, \infty, \infty]$	Pre-emptive strategy
3	$t_2 = t_1 \oplus x_4 // r_0$	3	$[0, 0, 2, 0, \infty, 0, \infty, \infty]$	Pre-emptive strategy
4	$g_0 = \text{AND}(t_2, t_0) // y_0$	4	$[0, 0, 2, 0, \infty, 0, 0, \infty]$	New pre-emptive strategy
5	$t_3 = x_3 \oplus x_4$	1	$[0, 0, 1, 0, \infty, 0, 0, \infty]$	
6	$t_4 = t_0 \oplus t_3 // r_2$	2	$[0, 0, 0, 0, \infty, 0, 0, \infty]$	Pre-emptive strategy
7	$g_1 = \text{AND}(t_4, x_2)$	3	$[0, 0, 0, 0, 0, 0, 0, \infty]$	New pre-emptive strategy
8	$g_2 = \text{AND}(g_1, t_1) // y_1$	4	$[0, 0, 0, 0, 0, 0, 0, 0]$	New pre-emptive strategy

Table 10 demonstrates that t_2 and t_4 have the same values, $x_0 \oplus x_1 \oplus x_3 \oplus x_4$. Nevertheless, eBPD fails to achieve the depth limit of r_2 due to the high depth of t_2 ; hence, it implements t_4 with a lower depth. In this example, the two target nodes r_0 and r_2 have the same value and different depths, yet this limitation persists even if the target node values all differ. To address this limitation, we considered target nodes with the same value as one and adopted a low depth. Then, after eBPD was completed, we rearranged the nodes of the implementation based on depth to discard duplicate XORs with high depth.

C Novel AES, Ascon, Dillon, and Cube x^3 S-Box Circuits

This section presents the proposed AES, Ascon, Dillon, and the cube x^3 S-box circuits. For all i and j , x_i and y_i represent the input and output nodes of the circuit, respectively, t_j denotes a temporary node, and g_i is the output of the i -th nonlinear gate. In the listings, XOR and XNOR gates are represented using $\oplus$ and $\ominus$, respectively, whereas nonlinear gates are expressed explicitly as AND, OR, NAND, or NOR.

Listing 1: New AES S-box circuit ($\mathcal{D}$: 14, $\mathcal{AD}$: 4, $\#\mathcal{NL}$: 33, $\#\mathcal{L}$: 95, $\#\mathcal{G}$: 128)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_1 \oplus x_3$	3	$t_{29} = t_0 \oplus g_2$	8	$g_{13} = \text{NAND}(t_{47}, t_{41})$	11	$g_{29} = \text{NAND}(t_4, t_{59})$
1	$t_1 = x_1 \oplus x_2$	3	$t_{30} = t_{21} \ominus t_9$	8	$g_{14} = \text{NAND}(t_{44}, t_{48})$	11	$g_{30} = \text{NAND}(t_{30}, t_{59})$
1	$t_2 = x_3 \oplus x_4$	3	$t_{31} = g_1 \ominus t_{11}$	9	$t_{51} = g_{11} \oplus g_{13}$	11	$g_{31} = \text{NAND}(t_5, t_{58})$
1	$t_3 = x_0 \oplus x_3$	3	$g_3 = \text{NAND}(t_{12}, t_{20})$	9	$t_{52} = g_{12} \oplus g_{14}$	11	$g_{32} = \text{NAND}(t_{27}, t_{58})$
1	$t_4 = x_3 \oplus x_5$	3	$g_4 = \text{NAND}(t_{15}, t_{19})$	9	$t_{53} = t_{42} \oplus g_{11}$	12	$t_{67} = g_{29} \ominus g_{31}$
1	$t_5 = x_0 \oplus x_5$	3	$g_5 = \text{NAND}(t_{13}, t_{22})$	9	$t_{54} = t_{42} \ominus g_{13}$	12	$t_{68} = g_{30} \ominus g_{32}$
1	$t_6 = x_0 \oplus x_7$	3	$g_6 = \text{NAND}(t_{16}, t_{14})$	9	$t_{55} = t_{43} \ominus g_{14}$	12	$t_{69} = g_{24} \ominus t_{60}$
1	$t_7 = x_0 \ominus x_6$	3	$g_7 = \text{NOR}(t_8, t_{23})$	9	$t_{56} = t_{43} \ominus g_{12}$	12	$t_{70} = t_{63} \ominus t_{64}$
1	$t_8 = x_3 \ominus x_6$	4	$t_{32} = t_{25} \oplus t_{21}$	10	$t_{57} = t_{54} \oplus t_{55}$	12	$t_{71} = t_{64} \ominus t_{65}$
1	$t_9 = x_2 \ominus x_6$	4	$t_{33} = t_{27} \oplus t_{30}$	10	$t_{58} = t_{51} \oplus t_{52}$	12	$t_{72} = g_{28} \ominus g_{32}$
1	$t_{10} = x_5 \ominus x_7$	4	$t_{34} = g_3 \oplus g_7$	10	$t_{59} = t_{53} \ominus t_{56}$	12	$t_{73} = g_{18} \ominus t_{60}$
1	$t_{11} = x_4 \ominus x_5$	4	$t_{35} = t_3 \oplus t_{28}$	10	$g_{15} = \text{NAND}(t_{17}, t_{54})$	12	$t_{74} = t_{65} \ominus t_{61}$
2	$t_{12} = x_1 \oplus t_6$	4	$t_{36} = g_4 \ominus g_6$	10	$g_{16} = \text{NAND}(t_{40}, t_{54})$	12	$t_{75} = g_{27} \ominus g_{29}$
2	$t_{13} = x_7 \oplus t_1$	4	$t_{37} = g_4 \ominus t_{29}$	10	$g_{17} = \text{NAND}(t_{13}, t_{51})$	12	$t_{76} = t_{64} \ominus t_{66}$
2	$t_{14} = x_4 \oplus t_9$	4	$t_{38} = g_3 \ominus t_{28}$	10	$g_{18} = \text{NAND}(t_{24}, t_{51})$	13	$t_{77} = t_{67} \ominus t_{62}$
2	$t_{15} = x_2 \oplus t_{10}$	4	$t_{39} = g_5 \ominus t_{31}$	10	$g_{19} = \text{NAND}(t_{25}, t_{55})$	13	$t_{78} = g_{15} \ominus t_{67}$
2	$t_{16} = t_1 \oplus t_{10}$	4	$g_8 = \text{NAND}(t_{24}, t_{26})$	10	$g_{20} = \text{NAND}(x_7, t_{55})$	13	$t_{79} = g_{20} \ominus t_{69}$
2	$t_{17} = t_6 \ominus t_1$	5	$t_{40} = x_7 \oplus t_{33}$	10	$g_{21} = \text{NAND}(t_{26}, t_{52})$	13	$t_{80} = t_{68} \ominus t_{64}$
2	$t_{18} = t_2 \ominus t_7$	5	$t_{41} = t_{37} \oplus t_{35}$	10	$g_{22} = \text{NAND}(t_{32}, t_{52})$	13	$t_{81} = t_{63} \ominus t_{68}$
2	$t_{19} = t_1 \ominus t_3$	5	$t_{42} = g_8 \oplus t_{38}$	10	$g_{23} = \text{NAND}(t_{21}, t_{56})$	13	$t_{82} = t_{69} \ominus t_{71}$
2	$t_{20} = t_1 \ominus t_4$	5	$t_{43} = t_{36} \oplus t_{39}$	10	$g_{24} = \text{NAND}(t_{18}, t_{56})$	13	$t_{83} = t_{72} \ominus t_{73}$
2	$t_{21} = t_7 \ominus t_4$	5	$t_{44} = t_{34} \ominus t_{36}$	10	$g_{25} = \text{NOR}(t_7, t_{53})$	13	$t_{84} = t_{72} \ominus t_{74}$
2	$t_{22} = t_8 \ominus t_{10}$	5	$t_{45} = t_{34} \ominus t_{39}$	10	$g_{26} = \text{NOR}(t_{23}, t_{53})$	13	$t_{85} = t_{68} \ominus t_{75}$
2	$t_{23} = t_9 \ominus t_{11}$	6	$t_{46} = t_{41} \oplus t_{42}$	11	$t_{60} = g_{16} \oplus g_{26}$	13	$t_{86} = t_{67} \ominus t_{76}$
2	$g_0 = \text{NAND}(t_6, x_0)$	6	$g_9 = \text{NAND}(t_{44}, t_{41})$	11	$t_{61} = g_{25} \oplus g_{24}$	14	$y_0 = t_{77} \oplus t_{80}$
2	$g_1 = \text{NAND}(t_0, t_1)$	6	$g_{10} = \text{NAND}(t_{43}, t_{42})$	11	$t_{62} = g_{19} \ominus g_{22}$	14	$y_1 = t_{77} \ominus t_{81}$
2	$g_2 = \text{NAND}(t_2, t_7)$	7	$t_{47} = g_{10} \oplus t_{45}$	11	$t_{63} = g_{20} \ominus g_{21}$	14	$y_2 = t_{84} \oplus t_{85}$
3	$t_{24} = x_6 \oplus t_{13}$	7	$t_{48} = t_{46} \oplus g_{10}$	11	$t_{64} = g_{16} \ominus g_{17}$	14	$y_3 = t_{77} \oplus t_{70}$
3	$t_{25} = x_3 \oplus t_{13}$	7	$t_{49} = g_9 \ominus t_{42}$	11	$t_{65} = g_{18} \ominus g_{21}$	14	$y_4 = t_{77} \oplus t_{79}$
3	$t_{26} = x_7 \oplus t_{18}$	7	$t_{50} = g_9 \ominus t_{43}$	11	$t_{66} = g_{19} \ominus g_{23}$	14	$y_5 = t_{84} \oplus t_{86}$
3	$t_{27} = t_1 \oplus t_{18}$	8	$g_{11} = \text{NAND}(t_{46}, t_{50})$	11	$g_{27} = \text{NAND}(t_{33}, t_{57})$	14	$y_6 = t_{78} \ominus t_{83}$
3	$t_{28} = g_0 \oplus g_1$	8	$g_{12} = \text{NAND}(t_{49}, t_{45})$	11	$g_{28} = \text{NAND}(t_{33}, t_{57})$	14	$y_7 = t_{78} \ominus t_{82}$

Listing 2: New AES S-box circuit ($\mathcal{D}$: 13, $\mathcal{AD}$: 3, $\#\mathcal{NL}$: 42, $\#\mathcal{L}$: 94, $\#\mathcal{G}$: 136)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_0 \oplus x_6$	4	$g_5 = \text{NAND}(t_0, t_{24})$	7	$g_{24} = \text{NAND}(t_{13}, t_{44})$	10	$g_{30} = \text{NAND}(t_{72}, t_{60})$
1	$t_1 = x_3 \oplus x_5$	4	$g_6 = \text{NAND}(t_{11}, t_{20})$	7	$g_{25} = \text{NAND}(t_{21}, t_{45})$	10	$g_{31} = \text{NAND}(t_{73}, t_{61})$
1	$t_2 = x_0 \oplus x_3$	4	$g_7 = \text{NAND}(t_3, t_{21})$	7	$g_{26} = \text{NAND}(t_{16}, t_{43})$	10	$g_{32} = \text{NAND}(t_{74}, t_{63})$
1	$t_3 = x_0 \oplus x_5$	4	$g_8 = \text{NAND}(t_{22}, t_{23})$	7	$g_{27} = \text{NAND}(t_{38}, t_{40})$	10	$g_{33} = \text{NAND}(t_{72}, t_{65})$
1	$t_4 = x_0 \oplus x_2$	5	$t_{30} = g_6 \oplus g_7$	7	$g_{28} = \text{NAND}(t_{39}, t_{41})$	10	$g_{34} = \text{NAND}(t_{73}, t_{64})$
1	$t_5 = x_4 \oplus x_5$	5	$t_{31} = g_5 \oplus t_{26}$	7	$g_{29} = \text{NAND}(t_{37}, t_{42})$	10	$g_{35} = \text{NAND}(t_{74}, t_{62})$
1	$t_6 = x_1 \oplus x_7$	5	$t_{32} = g_7 \oplus t_{27}$	8	$t_{46} = t_{45} \oplus g_{28}$	10	$g_{36} = \text{NAND}(t_{72}, t_{66})$
1	$t_7 = x_1 \oplus x_6$	5	$t_{33} = g_0 \oplus g_8$	8	$t_{47} = t_{43} \oplus g_{27}$	10	$g_{37} = \text{NAND}(t_{73}, t_{67})$
1	$t_8 = x_2 \oplus x_3$	5	$t_{34} = g_6 \oplus t_{27}$	8	$t_{48} = g_{10} \oplus g_{17}$	10	$g_{38} = \text{NAND}(t_{74}, t_{69})$
1	$t_9 = x_4 \oplus x_6$	5	$t_{35} = g_3 \oplus g_8$	8	$t_{49} = g_9 \oplus g_{16}$	10	$g_{39} = \text{NAND}(t_{72}, t_{71})$
2	$t_{10} = t_0 \oplus t_1$	5	$t_{36} = g_5 \oplus t_{29}$	8	$t_{50} = g_{11} \oplus g_{15}$	10	$g_{40} = \text{NAND}(t_{73}, t_{70})$
2	$t_{11} = t_4 \oplus t_6$	6	$t_{37} = t_{25} \oplus t_{30}$	8	$t_{51} = g_{13} \oplus g_{17}$	10	$g_{41} = \text{NAND}(t_{74}, t_{68})$
2	$t_{12} = x_6 \oplus t_5$	6	$t_{38} = t_{30} \oplus t_{31}$	8	$t_{52} = g_{12} \oplus g_{16}$	11	$t_{75} = g_{30} \oplus g_{31}$
2	$t_{13} = t_5 \oplus t_7$	6	$t_{39} = t_{25} \oplus t_{31}$	8	$t_{53} = g_{14} \oplus g_{15}$	11	$t_{76} = g_{36} \oplus g_{38}$
2	$t_{14} = t_6 \oplus t_8$	6	$t_{40} = t_{28} \oplus t_{32}$	8	$t_{54} = g_{19} \oplus g_{26}$	11	$t_{77} = g_{40} \oplus g_{41}$
2	$t_{15} = x_1 \oplus t_8$	6	$t_{41} = t_{28} \oplus t_{33}$	8	$t_{55} = g_{18} \oplus g_{25}$	11	$t_{78} = g_{33} \oplus g_{35}$
2	$t_{16} = t_1 \oplus t_4$	6	$t_{42} = t_{32} \oplus t_{33}$	8	$t_{56} = g_{20} \oplus g_{24}$	11	$t_{79} = g_{31} \oplus g_{32}$
2	$t_{17} = x_2 \oplus t_6$	6	$t_{43} = t_{34} \oplus t_{35}$	8	$t_{57} = g_{22} \oplus g_{26}$	11	$t_{80} = g_{37} \oplus g_{38}$
2	$t_{18} = t_2 \oplus t_9$	6	$t_{44} = t_{34} \oplus t_{36}$	8	$t_{58} = g_{21} \oplus g_{25}$	11	$t_{81} = g_{39} \oplus g_{40}$
2	$g_0 = \text{NAND}(t_1, t_4)$	6	$t_{45} = t_{35} \oplus t_{36}$	8	$t_{59} = g_{23} \oplus g_{24}$	11	$t_{82} = g_{34} \oplus g_{35}$
3	$t_{19} = x_6 \oplus t_{17}$	7	$g_9 = \text{NAND}(t_{14}, t_{38})$	9	$t_{60} = t_{49} \oplus t_{50}$	12	$t_{83} = t_{77} \oplus t_{79}$
3	$t_{20} = t_6 \oplus t_{12}$	7	$g_{10} = \text{NAND}(t_{22}, t_{39})$	9	$t_{61} = t_{48} \oplus t_{50}$	12	$t_{84} = t_{75} \oplus t_{78}$
3	$t_{21} = t_{16} \oplus t_{13}$	7	$g_{11} = \text{NAND}(t_{10}, t_{37})$	9	$t_{62} = t_{51} \oplus t_{52}$	12	$t_{85} = t_{76} \oplus t_{77}$
3	$t_{22} = t_{10} \oplus t_{14}$	7	$g_{12} = \text{NAND}(t_{11}, t_{40})$	9	$t_{63} = t_{48} \oplus t_{49}$	12	$t_{86} = t_{76} \oplus t_{81}$
3	$t_{23} = x_7 \oplus t_{18}$	7	$g_{13} = \text{NAND}(t_{19}, t_{41})$	9	$t_{64} = t_{51} \oplus t_{53}$	12	$t_{87} = t_{79} \oplus t_{80}$
3	$t_{24} = x_2 \oplus t_{12}$	7	$g_{14} = \text{NAND}(t_0, t_{42})$	9	$t_{65} = t_{52} \oplus t_{53}$	12	$t_{88} = t_{81} \oplus t_{82}$
3	$g_1 = \text{NAND}(t_{14}, x_7)$	7	$g_{15} = \text{NAND}(t_2, t_{44})$	9	$t_{66} = t_{55} \oplus t_{56}$	12	$y_0 = t_{83}$
3	$g_2 = \text{NAND}(t_{18}, t_5)$	7	$g_{16} = \text{NAND}(t_3, t_{45})$	9	$t_{67} = t_{54} \oplus t_{56}$	12	$y_1 = t_{87}$
3	$g_3 = \text{NOR}(t_{17}, x_6)$	7	$g_{17} = \text{NAND}(t_1, t_{43})$	9	$t_{68} = t_{57} \oplus t_{58}$	12	$y_2 = t_{76} \oplus t_{78}$
3	$g_4 = \text{NOR}(t_2, t_{13})$	7	$g_{18} = \text{NAND}(x_7, t_{38})$	9	$t_{69} = t_{54} \oplus t_{55}$	13	$y_3 = t_{80} \oplus t_{83}$
4	$t_{25} = g_0 \oplus g_3$	7	$g_{19} = \text{NAND}(t_{23}, t_{39})$	9	$t_{70} = t_{57} \oplus t_{59}$	13	$y_4 = t_{87} \oplus t_{86}$
4	$t_{26} = g_4 \oplus t_{15}$	7	$g_{20} = \text{NAND}(t_{18}, t_{37})$	9	$t_{71} = t_{58} \oplus t_{59}$	13	$y_5 = t_{84} \oplus t_{85}$
4	$t_{27} = t_4 \oplus g_1$	7	$g_{21} = \text{NAND}(t_{20}, t_{40})$	9	$t_{72} = g_{29} \oplus t_{46}$	12	$y_6 = t_{88}$
4	$t_{28} = g_4 \oplus g_2$	7	$g_{22} = \text{NAND}(t_{17}, t_{41})$	9	$t_{73} = g_{29} \oplus t_{47}$	13	$y_7 = t_{85} \oplus t_{88}$
4	$t_{29} = t_{15} \oplus g_2$	7	$g_{23} = \text{NAND}(t_{24}, t_{42})$	9	$t_{74} = t_{46} \oplus t_{47}$		

Listing 3: New AES S-box circuit ($\mathcal{D}$: 12, $\mathcal{AD}$: 3, $\#\mathcal{NL}$: 42, $\#\mathcal{L}$: 107, $\#\mathcal{G}$: 149)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_2 \oplus x_4$	3	$g_2 = \text{NAND}(t_{15}, t_8)$	6	$g_{17} = \text{NAND}(t_6, t_{50})$	8	$t_{84} = t_{71} \oplus t_{72}$
1	$t_1 = x_1 \oplus x_4$	3	$g_3 = \text{NAND}(t_{18}, t_{23})$	6	$g_{18} = \text{NAND}(x_7, t_{52})$	8	$t_{85} = g_{29} \oplus t_{59}$
1	$t_2 = x_2 \oplus x_7$	3	$g_4 = \text{NAND}(t_{16}, t_{24})$	6	$g_{19} = \text{NAND}(t_{29}, t_{55})$	8	$t_{86} = g_{29} \oplus t_{60}$
1	$t_3 = x_0 \oplus x_6$	3	$g_5 = \text{NAND}(t_{19}, t_{20})$	6	$g_{20} = \text{NAND}(t_{34}, t_{54})$	8	$t_{87} = t_{59} \oplus t_{60}$
1	$t_4 = x_0 \oplus x_3$	3	$g_6 = \text{NAND}(t_{25}, t_{22})$	6	$g_{21} = \text{NAND}(t_{38}, t_{57})$	9	$g_{30} = \text{NAND}(t_{85}, t_{73})$
1	$t_5 = x_0 \oplus x_5$	3	$g_7 = \text{NAND}(t_{26}, t_{13})$	6	$g_{22} = \text{NAND}(t_{17}, t_{51})$	9	$g_{31} = \text{NAND}(t_{86}, t_{74})$
1	$t_6 = x_3 \oplus x_5$	3	$g_8 = \text{NAND}(t_{27}, t_{21})$	6	$g_{23} = \text{NAND}(t_{33}, t_{53})$	9	$g_{32} = \text{NAND}(t_{87}, t_{76})$
1	$t_7 = x_3 \oplus x_7$	4	$t_{36} = t_{14} \oplus t_{32}$	6	$g_{24} = \text{NAND}(t_{39}, t_{58})$	9	$g_{33} = \text{NAND}(t_{85}, t_{78})$
1	$t_8 = x_1 \oplus x_2$	4	$t_{37} = t_3 \oplus t_{28}$	6	$g_{25} = \text{NAND}(t_{35}, t_{56})$	9	$g_{34} = \text{NAND}(t_{86}, t_{77})$
1	$t_9 = x_2 \oplus x_6$	4	$t_{38} = t_{17} \oplus t_{33}$	6	$g_{26} = \text{NAND}(t_{40}, t_{50})$	9	$g_{35} = \text{NAND}(t_{87}, t_{75})$
1	$t_{10} = x_0 \oplus x_1$	4	$t_{39} = t_8 \oplus t_{33}$	6	$g_{27} = \text{NAND}(t_{52}, t_{57})$	9	$g_{36} = \text{NAND}(t_{85}, t_{79})$
1	$t_{11} = x_1 \oplus x_6$	4	$t_{40} = t_{32} \oplus t_9$	6	$g_{28} = \text{NAND}(t_{55}, t_{51})$	9	$g_{37} = \text{NAND}(t_{86}, t_{80})$
1	$t_{12} = x_5 \oplus x_7$	4	$t_{41} = g_8 \oplus t_{30}$	6	$g_{29} = \text{NAND}(t_{54}, t_{53})$	9	$g_{38} = \text{NAND}(t_{87}, t_{82})$
2	$t_{13} = x_2 \oplus t_7$	4	$t_{42} = g_4 \oplus g_5$	7	$t_{59} = t_{56} \oplus g_{28}$	9	$g_{39} = \text{NAND}(t_{85}, t_{84})$
2	$t_{14} = t_7 \oplus t_8$	4	$t_{43} = g_2 \oplus g_6$	7	$t_{60} = t_{50} \oplus g_{27}$	9	$g_{40} = \text{NAND}(t_{86}, t_{83})$
2	$t_{15} = t_0 \oplus t_4$	4	$t_{44} = g_2 \oplus g_3$	7	$t_{61} = g_{10} \oplus g_{17}$	9	$g_{41} = \text{NAND}(t_{87}, t_{81})$
2	$t_{16} = t_0 \oplus t_6$	4	$t_{45} = g_5 \oplus t_{31}$	7	$t_{62} = g_9 \oplus g_{16}$	10	$t_{88} = g_{30} \oplus g_{31}$
2	$t_{17} = x_1 \oplus t_2$	4	$t_{46} = g_7 \oplus t_{30}$	7	$t_{63} = g_{11} \oplus g_{15}$	10	$t_{89} = g_{36} \oplus g_{38}$
2	$t_{18} = t_4 \oplus t_9$	4	$t_{47} = g_3 \oplus g_6$	7	$t_{64} = g_{13} \oplus g_{17}$	10	$t_{90} = g_{40} \oplus g_{41}$
2	$t_{19} = t_7 \oplus t_{10}$	4	$t_{48} = g_4 \oplus t_{31}$	7	$t_{65} = g_{12} \oplus g_{16}$	10	$t_{91} = g_{33} \oplus g_{35}$
2	$t_{20} = x_6 \oplus t_0$	4	$t_{49} = g_7 \oplus g_8$	7	$t_{66} = g_{14} \oplus g_{15}$	10	$t_{92} = g_{31} \oplus g_{32}$
2	$t_{21} = x_4 \oplus t_2$	5	$t_{50} = t_{41} \oplus t_{43}$	7	$t_{67} = g_{19} \oplus g_{26}$	10	$t_{93} = g_{37} \oplus g_{38}$
2	$t_{22} = x_5 \oplus t_3$	5	$t_{51} = t_{45} \oplus t_{46}$	7	$t_{68} = g_{18} \oplus g_{25}$	10	$t_{94} = g_{39} \oplus g_{40}$
2	$t_{23} = x_4 \oplus t_5$	5	$t_{52} = t_{47} \oplus t_{48}$	7	$t_{69} = g_{20} \oplus g_{24}$	10	$t_{95} = g_{34} \oplus g_{35}$
2	$t_{24} = t_0 \oplus t_{10}$	5	$t_{53} = t_{41} \oplus t_{42}$	7	$t_{70} = g_{22} \oplus g_{26}$	11	$t_{96} = t_{90} \oplus t_{92}$
2	$t_{25} = t_7 \oplus t_{11}$	5	$t_{54} = t_{42} \oplus t_{43}$	7	$t_{71} = g_{21} \oplus g_{25}$	11	$t_{97} = t_{88} \oplus t_{91}$
2	$t_{26} = x_6 \oplus t_{12}$	5	$t_{55} = t_{44} \oplus t_{45}$	7	$t_{72} = g_{23} \oplus g_{24}$	11	$t_{98} = t_{89} \oplus t_{90}$
2	$t_{27} = t_{11} \oplus t_{12}$	5	$t_{56} = t_{44} \oplus t_{46}$	8	$t_{73} = t_{62} \oplus t_{63}$	11	$t_{99} = t_{89} \oplus t_{94}$
2	$g_0 = \text{NAND}(t_0, t_7)$	5	$t_{57} = t_{48} \oplus t_{49}$	8	$t_{74} = t_{61} \oplus t_{63}$	11	$t_{100} = t_{92} \oplus t_{93}$
2	$g_1 = \text{NAND}(t_1, t_2)$	5	$t_{58} = t_{47} \oplus t_{49}$	8	$t_{75} = t_{64} \oplus t_{65}$	11	$t_{101} = t_{94} \oplus t_{95}$
3	$t_{28} = t_{14} \oplus t_4$	6	$g_9 = \text{NAND}(t_{14}, t_{52})$	8	$t_{76} = t_{61} \oplus t_{62}$	11	$y_0 = t_{96}$
3	$t_{29} = t_{21} \oplus t_{18}$	6	$g_{10} = \text{NAND}(t_{36}, t_{55})$	8	$t_{77} = t_{64} \oplus t_{66}$	11	$y_1 = t_{100}$
3	$t_{30} = x_2 \oplus g_1$	6	$g_{11} = \text{NAND}(t_{32}, t_{54})$	8	$t_{78} = t_{65} \oplus t_{66}$	11	$y_2 = t_{89} \oplus t_{91}$
3	$t_{31} = x_5 \oplus g_0$	6	$g_{12} = \text{NAND}(t_{28}, t_{57})$	8	$t_{79} = t_{68} \oplus t_{69}$	12	$y_3 = t_{93} \oplus t_{96}$
3	$t_{32} = x_2 \oplus t_{22}$	6	$g_{13} = \text{NAND}(t_{37}, t_{51})$	8	$t_{80} = t_{67} \oplus t_{69}$	12	$y_4 = t_{100} \oplus t_{99}$
3	$t_{33} = x_5 \oplus t_{20}$	6	$g_{14} = \text{NAND}(t_3, t_{53})$	8	$t_{81} = t_{70} \oplus t_{71}$	12	$y_5 = t_{97} \oplus t_{98}$
3	$t_{34} = t_0 \oplus t_{18}$	6	$g_{15} = \text{NAND}(t_4, t_{58})$	8	$t_{82} = t_{67} \oplus t_{68}$	11	$y_6 = t_{101}$
3	$t_{35} = t_1 \oplus t_{18}$	6	$g_{16} = \text{NAND}(t_5, t_{56})$	8	$t_{83} = t_{70} \oplus t_{72}$	12	$y_7 = t_{98} \oplus t_{101}$

Listing 4: New Ascon S-box circuit ($\mathcal{D}$: 4, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 5, $\#\mathcal{L}$: 11, $\#\mathcal{G}$: 16)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_0 \oplus x_4$	1	$t_5 = x_1 \oplus 1$	2	$g_2 = \text{NAND}(t_1, t_2)$	3	$y_1 = t_0 \oplus g_2$
1	$t_1 = x_2 \oplus x_3$	1	$t_6 = x_0 \oplus 1$	2	$g_3 = \text{NAND}(x_4, t_4)$	3	$y_2 = g_3 \oplus t_2$
1	$t_2 = x_1 \oplus x_2$	2	$t_7 = x_3 \oplus t_6$	2	$g_4 = \text{NAND}(t_3, t_6)$	3	$y_3 = g_4 \oplus t_2$
1	$t_3 = x_3 \oplus x_4$	2	$g_0 = \text{NAND}(x_2, t_5)$	3	$t_8 = g_0 \oplus t_3$	3	$y_4 = g_1 \oplus t_7$
1	$t_4 = x_3 \oplus 1$	2	$g_1 = \text{NAND}(t_0, t_5)$	4	$y_0 = t_8 \oplus g_1$		

Listing 5: New Dillon S-box circuit ($\mathcal{D}$: 10, $\mathcal{AD}$: 3, $\#\mathcal{NL}$: 12, $\#\mathcal{L}$: 39, $\#\mathcal{G}$: 51)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_3 \oplus x_5$	4	$t_{11} = t_1 \oplus t_7$	6	$t_{22} = g_5 \oplus t_{17}$	9	$t_{30} = g_9 \oplus t_{28}$
1	$t_1 = x_3 \oplus x_4$	4	$t_{12} = t_8 \oplus t_9$	7	$t_{23} = t_0 \oplus t_{18}$	9	$t_{31} = g_9 \oplus g_8$
1	$t_2 = x_0 \oplus x_3$	4	$t_{13} = t_6 \oplus t_7$	7	$t_{24} = t_0 \oplus t_{20}$	9	$t_{32} = g_7 \oplus g_{11}$
1	$t_3 = x_0 \oplus x_4$	4	$t_{14} = g_2 \oplus t_7$	7	$t_{25} = g_4 \oplus t_{21}$	9	$t_{33} = x_4 \oplus g_8$
1	$t_4 = x_2 \oplus x_3$	4	$t_{15} = g_2 \oplus t_{10}$	7	$t_{26} = x_5 \oplus t_{18}$	9	$t_{34} = g_{10} \oplus t_{29}$
1	$t_5 = x_4 \oplus 1$	4	$g_3 = \text{NOR}(t_9, t_8)$	7	$t_{27} = g_5 \oplus t_{19}$	10	$y_0 = t_{31} \oplus t_{34}$
2	$t_6 = x_5 \oplus t_2$	5	$t_{16} = x_3 \oplus t_{14}$	7	$t_{28} = t_{21} \oplus t_{12}$	6	$y_1 = t_{22}$
2	$g_0 = \text{NAND}(x_3, t_5)$	5	$t_{17} = t_{10} \oplus t_{11}$	7	$g_6 = \text{NAND}(t_{20}, t_{22})$	10	$y_2 = g_9 \oplus t_{33}$
2	$g_1 = \text{NAND}(x_5, t_0)$	5	$g_4 = \text{NAND}(t_{13}, t_8)$	8	$t_{29} = x_5 \oplus g_6$	8	$y_3 = t_{26} \oplus t_{20}$
2	$g_2 = \text{NAND}(t_0, t_1)$	5	$g_5 = \text{NAND}(t_{14}, t_9)$	8	$g_7 = \text{AND}(t_{24}, t_{25})$	10	$y_4 = t_{33} \oplus t_{30}$
3	$t_7 = g_0 \oplus t_4$	6	$t_{18} = g_3 \oplus g_5$	8	$g_8 = \text{OR}(t_{28}, t_{20})$	10	$y_5 = t_{31} \oplus t_{32}$
3	$t_8 = g_1 \oplus t_3$	6	$t_{19} = t_{15} \oplus t_{16}$	8	$g_9 = \text{NOR}(t_{24}, t_{23})$		
3	$t_9 = g_2 \oplus t_6$	6	$t_{20} = g_4 \oplus t_{15}$	8	$g_{10} = \text{NOR}(t_{27}, t_{23})$		
3	$t_{10} = x_1 \oplus g_1$	6	$t_{21} = g_3 \oplus t_{16}$	8	$g_{11} = \text{NOR}(t_{22}, t_{26})$		

Listing 6: New Dillon S-box circuit ($\mathcal{D}$: 7, $\mathcal{AD}$: 2, $\#\mathcal{NL}$: 20, $\#\mathcal{L}$: 49, $\#\mathcal{G}$: 69)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_3 \oplus x_4$	2	$g_2 = \text{NAND}(t_0, t_1)$	4	$t_{26} = x_2 \oplus g_9$	5	$t_{40} = g_4 \oplus t_{26}$
1	$t_1 = x_3 \oplus x_5$	2	$g_3 = \text{NAND}(t_2, t_3)$	4	$t_{27} = t_{10} \oplus t_{23}$	5	$t_{41} = g_{13} \oplus t_{20}$
1	$t_2 = x_0 \oplus x_4$	2	$g_4 = \text{NAND}(t_4, t_0)$	4	$t_{28} = g_2 \oplus t_{16}$	5	$g_{14} = \text{NAND}(t_{19}, t_{28})$
1	$t_3 = x_1 \oplus x_4$	2	$g_5 = \text{OR}(x_5, t_5)$	4	$t_{29} = t_1 \oplus t_{22}$	5	$g_{15} = \text{NAND}(t_{19}, t_{29})$
1	$t_4 = x_4 \oplus x_5$	3	$t_{16} = t_3 \oplus t_{15}$	4	$t_{30} = g_1 \oplus g_6$	6	$t_{42} = g_9 \oplus t_{34}$
1	$t_5 = x_0 \oplus x_3$	3	$t_{17} = x_1 \oplus t_{15}$	4	$t_{31} = g_5 \oplus g_7$	6	$g_{16} = \text{NAND}(t_{32}, t_{37})$
1	$t_6 = x_1 \oplus x_5$	3	$t_{18} = g_3 \oplus t_4$	4	$g_{10} = \text{NAND}(t_{21}, t_7)$	6	$g_{17} = \text{NAND}(t_{39}, t_{33})$
1	$t_7 = x_1 \oplus x_2$	3	$t_{19} = g_2 \oplus t_{12}$	4	$g_{11} = \text{OR}(t_{16}, t_5)$	6	$g_{18} = \text{NAND}(t_{32}, t_{40})$
1	$g_0 = \text{NAND}(x_4, x_3)$	3	$t_{20} = t_0 \oplus g_3$	4	$g_{12} = \text{OR}(t_{17}, t_0)$	6	$g_{19} = \text{NAND}(t_{38}, t_{41})$
1	$g_1 = \text{NOR}(x_5, x_3)$	3	$t_{21} = t_1 \oplus t_{14}$	4	$g_{13} = \text{OR}(t_{17}, t_9)$	7	$y_0 = g_{17} \oplus t_{35}$
2	$t_8 = x_2 \oplus t_3$	3	$t_{22} = g_2 \oplus t_{11}$	5	$t_{32} = g_{11} \oplus t_{18}$	6	$y_1 = g_{14} \oplus t_{22}$
2	$t_9 = x_1 \oplus t_4$	3	$t_{23} = g_3 \oplus g_4$	5	$t_{33} = g_6 \oplus t_{25}$	7	$y_2 = g_{16} \oplus t_{42}$
2	$t_{10} = x_0 \oplus t_1$	3	$g_6 = \text{NAND}(t_{14}, t_{15})$	5	$t_{34} = t_2 \oplus t_{24}$	6	$y_3 = g_{15} \oplus t_{43}$
2	$t_{11} = g_1 \oplus t_3$	3	$g_7 = \text{NAND}(t_6, t_{15})$	5	$t_{35} = t_0 \oplus t_{30}$	7	$y_4 = g_{18} \oplus t_{27}$
2	$t_{12} = x_2 \oplus g_0$	3	$g_8 = \text{OR}(t_8, t_5)$	5	$t_{36} = x_5 \oplus t_{31}$	7	$y_5 = g_{19} \oplus t_{36}$
2	$t_{13} = g_1 \oplus t_5$	3	$g_9 = \text{OR}(t_9, t_{10})$	5	$t_{37} = g_{12} \oplus t_{24}$		
2	$t_{14} = t_5 \oplus t_7$	4	$t_{24} = x_2 \oplus t_{18}$	5	$t_{38} = g_5 \oplus t_{24}$		
2	$t_{15} = x_0 \oplus t_4$	4	$t_{25} = x_0 \oplus g_8$	5	$t_{39} = g_{10} \oplus t_{11}$		

Listing 7: New 7-bit x^3 S-box circuit ($\mathcal{D}$: 6, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 11, $\#\mathcal{L}$: 41, $\#\mathcal{G}$: 52)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_0 \oplus x_2$	2	$t_{13} = t_6 \oplus x_2$	3	$g_4 = \text{NAND}(t_4, t_{13})$	5	$t_{28} = t_{22} \oplus g_5$
1	$t_1 = x_1 \oplus x_5$	2	$t_{14} = t_1 \oplus x_0$	3	$g_5 = \text{NOR}(t_6, t_{11})$	5	$t_{29} = g_8 \oplus g_9$
1	$t_2 = x_2 \oplus x_5$	2	$t_{15} = x_3 \oplus t_2$	3	$g_6 = \text{NOR}(t_8, t_{12})$	5	$t_{30} = g_8 \oplus t_{21}$
1	$t_3 = x_4 \oplus x_6$	2	$g_0 = \text{NAND}(t_0, t_1)$	4	$t_{22} = x_2 \oplus g_3$	5	$t_{31} = g_4 \oplus g_7$
1	$t_4 = x_0 \oplus x_1$	2	$g_1 = \text{NAND}(t_2, t_3)$	4	$t_{23} = g_5 \oplus t_9$	5	$t_{32} = t_{22} \oplus t_{24}$
1	$t_5 = x_2 \oplus x_6$	2	$g_2 = \text{NOR}(x_1, t_6)$	4	$t_{24} = g_6 \oplus t_2$	5	$t_{33} = t_{23} \oplus t_{25}$
1	$t_6 = x_0 \oplus x_4$	3	$t_{16} = t_6 \oplus t_{10}$	4	$t_{25} = t_{17} \oplus t_{12}$	5	$y_0 = t_{22} \oplus t_{17}$
1	$t_7 = x_5 \oplus x_6$	3	$t_{17} = g_2 \oplus g_0$	4	$g_7 = \text{NAND}(t_{10}, t_{18})$	6	$y_1 = t_{31} \oplus t_{28}$
2	$t_8 = x_3 \oplus t_3$	3	$t_{18} = t_{13} \oplus t_7$	4	$g_8 = \text{NAND}(x_4, t_{19})$	6	$y_2 = t_{26} \oplus t_{32}$
2	$t_9 = x_6 \oplus t_4$	3	$t_{19} = t_{10} \oplus t_7$	4	$g_9 = \text{NOR}(t_5, t_{16})$	6	$y_3 = t_{29} \oplus t_{33}$
2	$t_{10} = x_3 \oplus t_4$	3	$t_{20} = x_4 \oplus t_{15}$	4	$g_{10} = \text{NOR}(t_{14}, t_{20})$	6	$y_4 = x_3 \oplus t_{30}$
2	$t_{11} = t_1 \oplus x_3$	3	$t_{21} = g_0 \oplus g_1$	5	$t_{26} = g_{10} \oplus t_{21}$	6	$y_5 = g_4 \oplus t_{26}$
2	$t_{12} = t_1 \oplus x_6$	3	$g_3 = \text{NAND}(x_3, t_9)$	5	$t_{27} = g_{10} \oplus g_0$	6	$y_6 = t_{23} \oplus t_{27}$

Listing 8: New 7-bit x^3 S-box circuit ($\mathcal{D}$: 5, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 18, $\#\mathcal{L}$: 40, $\#\mathcal{G}$: 58)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_0 \oplus x_1$	1	$g_0 = \text{NAND}(x_2, x_4)$	2	$g_6 = \text{NAND}(x_1, t_4)$	3	$g_{16} = \text{NAND}(t_{24}, t_{22})$
1	$t_1 = x_0 \oplus x_4$	2	$t_{17} = x_4 \oplus t_3$	2	$g_7 = \text{NAND}(t_5, t_4)$	4	$t_{35} = g_6 \oplus t_{28}$
1	$t_2 = x_1 \oplus x_5$	2	$t_{18} = x_0 \oplus t_8$	2	$g_8 = \text{NAND}(t_6, t_7)$	4	$t_{36} = g_{10} \oplus g_5$
1	$t_3 = x_2 \oplus x_5$	2	$t_{19} = x_0 \oplus t_2$	2	$g_9 = \text{NAND}(t_8, t_3)$	4	$t_{37} = g_4 \oplus g_{11}$
1	$t_4 = x_3 \oplus x_4$	2	$t_{20} = x_4 \oplus t_2$	3	$t_{28} = x_6 \oplus g_1$	4	$t_{38} = g_7 \oplus t_{29}$
1	$t_5 = x_0 \oplus x_2$	2	$t_{21} = x_1 \oplus g_0$	3	$t_{29} = g_1 \oplus g_3$	4	$t_{39} = g_8 \oplus t_{30}$
1	$t_6 = x_1 \oplus x_2$	2	$t_{22} = x_4 \oplus t_6$	3	$t_{30} = g_1 \oplus t_{11}$	4	$t_{40} = g_{10} \oplus t_{31}$
1	$t_7 = x_0 \oplus x_5$	2	$t_{23} = x_6 \oplus t_1$	3	$t_{31} = g_0 \oplus g_3$	4	$t_{41} = g_8 \oplus t_{32}$
1	$t_8 = x_1 \oplus x_6$	2	$t_{24} = t_1 \oplus t_{11}$	3	$t_{32} = g_0 \oplus g_2$	4	$g_{17} = \text{NAND}(t_{34}, t_{27})$
1	$t_9 = x_0 \oplus x_3$	2	$t_{25} = t_0 \oplus 1$	3	$t_{33} = g_9 \oplus t_{21}$	5	$y_0 = g_{17} \oplus t_{40}$
1	$t_{10} = x_3 \oplus x_5$	2	$t_{26} = t_3 \oplus 1$	3	$t_{34} = t_{17} \oplus 1$	5	$y_1 = t_{36} \oplus t_{37}$
1	$t_{11} = x_5 \oplus x_6$	2	$t_{27} = t_6 \oplus 1$	3	$g_{10} = \text{NAND}(t_{10}, t_{25})$	5	$y_2 = g_{16} \oplus t_{35}$
1	$t_{12} = x_2 \oplus 1$	2	$g_1 = \text{NAND}(x_5, t_{12})$	3	$g_{11} = \text{NAND}(t_{26}, t_{16})$	5	$y_3 = g_{12} \oplus t_{38}$
1	$t_{13} = x_4 \oplus 1$	2	$g_2 = \text{NAND}(x_3, t_{13})$	3	$g_{12} = \text{NAND}(x_1, t_{23})$	5	$y_4 = g_{13} \oplus t_{41}$
1	$t_{14} = x_6 \oplus 1$	2	$g_3 = \text{NAND}(x_3, t_{14})$	3	$g_{13} = \text{NAND}(t_{17}, t_{18})$	4	$y_5 = g_{14} \oplus t_{33}$
1	$t_{15} = x_6 \oplus 1$	2	$g_4 = \text{NAND}(t_0, t_{15})$	3	$g_{14} = \text{NAND}(x_3, t_{19})$	5	$y_6 = g_{15} \oplus t_{39}$
1	$t_{16} = x_3 \oplus 1$	2	$g_5 = \text{NAND}(t_1, t_2)$	3	$g_{15} = \text{NAND}(t_9, t_{20})$		

Listing 9: New 7-bit x^3 S-box circuit ($\mathcal{D}$: 5, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 20, $\#\mathcal{L}$: 37, $\#\mathcal{G}$: 57)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_1 \oplus x_5$	1	$g_1 = \text{NAND}(x_0, x_3)$	2	$g_{12} = \text{OR}(t_8, t_0)$	3	$g_{19} = \text{NAND}(t_7, t_{16})$
1	$t_1 = x_0 \oplus x_6$	2	$t_{15} = x_4 \oplus t_1$	2	$g_{13} = \text{OR}(t_9, x_4)$	4	$t_{29} = g_8 \odot t_{21}$
1	$t_2 = x_3 \oplus x_4$	2	$t_{16} = t_3 \oplus t_5$	2	$g_{14} = \text{OR}(t_9, t_1)$	4	$t_{30} = g_7 \odot t_{22}$
1	$t_3 = x_1 \oplus x_4$	2	$t_{17} = x_2 \oplus t_1$	3	$t_{20} = x_6 \oplus g_3$	4	$t_{31} = g_8 \odot t_{20}$
1	$t_4 = x_0 \oplus x_1$	2	$t_{18} = x_6 \odot t_9$	3	$t_{21} = g_4 \odot g_0$	4	$t_{32} = g_{13} \odot t_{23}$
1	$t_5 = x_2 \oplus t_3$	2	$t_{19} = x_2 \odot t_3$	3	$t_{22} = g_3 \odot g_1$	4	$t_{33} = g_{11} \odot t_{24}$
1	$t_6 = x_0 \oplus x_5$	2	$g_2 = \text{NAND}(x_1, t_{11})$	3	$t_{23} = g_2 \odot g_5$	4	$y_0 = t_{27} \oplus t_{28}$
1	$t_7 = x_0 \oplus x_3$	2	$g_3 = \text{NAND}(x_1, t_{12})$	3	$t_{24} = g_4 \odot t_{10}$	5	$y_1 = g_{15} \oplus t_{29}$
1	$t_8 = x_1 \odot x_2$	2	$g_4 = \text{NAND}(x_2, t_{12})$	3	$t_{25} = g_{14} \odot g_9$	5	$y_2 = g_{17} \oplus t_{31}$
1	$t_9 = x_1 \odot x_3$	2	$g_5 = \text{NAND}(x_2, t_{13})$	3	$t_{26} = g_7 \odot g_{10}$	4	$y_3 = t_{25} \oplus t_{26}$
1	$t_{10} = x_2 \odot x_6$	2	$g_6 = \text{NAND}(t_2, t_{14})$	3	$t_{27} = g_2 \odot g_{14}$	5	$y_4 = g_{18} \oplus t_{32}$
1	$t_{11} = x_6 \oplus 1$	2	$g_7 = \text{NAND}(x_2, t_2)$	3	$t_{28} = g_{12} \odot g_6$	5	$y_5 = g_{16} \oplus t_{30}$
1	$t_{12} = x_3 \oplus 1$	2	$g_8 = \text{NAND}(t_3, t_0)$	3	$g_{15} = \text{NAND}(t_4, t_{18})$	5	$y_6 = g_{19} \oplus t_{33}$
1	$t_{13} = x_0 \oplus 1$	2	$g_9 = \text{NAND}(t_4, x_4)$	3	$g_{16} = \text{NAND}(t_5, t_{18})$		
1	$t_{14} = x_1 \oplus 1$	2	$g_{10} = \text{NAND}(x_5, t_5)$	3	$g_{17} = \text{NAND}(t_{15}, t_{19})$		
1	$g_0 = \text{NAND}(x_3, x_5)$	2	$g_{11} = \text{NAND}(x_1, t_6)$	3	$g_{18} = \text{NAND}(t_{16}, t_{17})$		

Listing 10: New 6-bit x^3 S-box circuit ($\mathcal{D}$: 6, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 8, $\#\mathcal{L}$: 28, $\#\mathcal{G}$: 36)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_2 \oplus x_3$	2	$t_9 = x_2 \odot t_4$	3	$g_4 = \text{NOR}(t_1, t_9)$	5	$t_{19} = g_1 \odot g_6$
1	$t_1 = x_0 \oplus x_1$	2	$g_0 = \text{NOR}(x_0, t_0)$	4	$t_{14} = x_3 \oplus g_2$	5	$t_{20} = g_7 \odot t_{14}$
1	$t_2 = x_4 \oplus x_5$	3	$t_{10} = t_2 \oplus t_9$	4	$t_{15} = x_2 \oplus g_3$	5	$t_{21} = g_5 \odot t_{15}$
1	$t_3 = x_3 \oplus t_0$	3	$t_{11} = t_3 \oplus t_8$	4	$t_{16} = g_0 \odot g_2$	6	$y_0 = t_{17} \oplus t_{12}$
1	$t_4 = x_0 \oplus x_4$	3	$t_{12} = g_0 \oplus t_6$	4	$g_5 = \text{NAND}(t_4, t_{11})$	6	$y_1 = t_0 \oplus t_{19}$
2	$t_5 = x_4 \oplus t_1$	3	$t_{13} = t_7 \odot t_4$	4	$g_6 = \text{NAND}(t_8, t_{13})$	4	$y_2 = g_0 \oplus g_4$
2	$t_6 = x_1 \oplus t_2$	3	$g_1 = \text{NAND}(t_2, t_5)$	4	$g_7 = \text{NOR}(x_4, t_{10})$	6	$y_3 = t_{17} \oplus t_{20}$
2	$t_7 = t_0 \oplus t_1$	3	$g_2 = \text{NAND}(x_3, t_6)$	5	$t_{17} = g_1 \odot g_5$	6	$y_4 = t_9 \oplus t_{18}$
2	$t_8 = x_0 \oplus t_2$	3	$g_3 = \text{NOR}(t_3, t_7)$	5	$t_{18} = g_6 \odot t_{16}$	6	$y_5 = t_{18} \oplus t_{21}$

Listing 11: New 6-bit x^3 S-box circuit ($\mathcal{D}$: 5, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 9, $\#\mathcal{L}$: 27, $\#\mathcal{G}$: 36)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_3 \oplus x_4$	2	$t_9 = x_0 \oplus g_0$	3	$t_{16} = g_3 \odot g_2$	4	$t_{21} = t_4 \odot t_{16}$
1	$t_1 = x_2 \oplus x_4$	2	$t_{10} = x_1 \odot t_2$	3	$g_4 = \text{NAND}(t_8, t_{10})$	5	$y_0 = t_{14} \oplus t_{19}$
1	$t_2 = x_0 \oplus x_2$	2	$t_{11} = t_1 \odot t_6$	3	$g_5 = \text{NAND}(t_2, t_{11})$	4	$y_1 = g_7 \oplus t_{15}$
1	$t_3 = x_0 \oplus x_5$	2	$t_{12} = t_2 \odot t_6$	3	$g_6 = \text{NAND}(t_7, t_{12})$	4	$y_2 = t_{17}$
1	$t_4 = x_2 \odot x_3$	2	$t_{13} = t_3 \odot t_6$	3	$g_7 = \text{NAND}(t_{12}, t_3)$	4	$y_3 = g_8 \oplus t_{15}$
1	$t_5 = x_2 \odot x_5$	2	$g_1 = \text{NAND}(t_0, t_1)$	3	$g_8 = \text{NAND}(t_0, t_{13})$	5	$y_4 = t_{20} \oplus t_{21}$
1	$t_6 = x_1 \odot x_3$	2	$g_2 = \text{NAND}(x_0, t_5)$	4	$t_{17} = g_1 \oplus g_6$	5	$y_5 = t_{17} \oplus t_{18}$
1	$g_0 = \text{NAND}(x_1, x_5)$	2	$g_3 = \text{OR}(t_4, x_4)$	4	$t_{18} = g_4 \oplus g_5$		
2	$t_7 = x_1 \oplus t_0$	3	$t_{14} = x_3 \oplus t_9$	4	$t_{19} = g_5 \odot g_6$		
2	$t_8 = x_4 \oplus t_3$	3	$t_{15} = g_3 \odot t_9$	4	$t_{20} = g_7 \odot g_8$		

Listing 12: New 6-bit x^3 S-box circuit ($\mathcal{D}$: 4, $\mathcal{AD}$: 1, $\#\mathcal{NL}$: 11, $\#\mathcal{L}$: 23, $\#\mathcal{G}$: 34)

Depth	Operation	Depth	Operation	Depth	Operation	Depth	Operation
1	$t_0 = x_1 \oplus x_2$	2	$t_9 = x_1 \odot t_4$	2	$g_7 = \text{OR}(t_6, t_3)$	3	$g_{10} = \text{NAND}(t_8, t_{10})$
1	$t_1 = x_4 \oplus x_5$	2	$t_{10} = x_3 \odot t_1$	3	$t_{11} = g_0 \oplus t_3$	4	$y_0 = t_{16} \oplus t_{11}$
1	$t_2 = x_2 \oplus x_3$	2	$g_0 = \text{NAND}(t_0, t_1)$	3	$t_{12} = g_3 \odot g_4$	4	$y_1 = t_{12} \oplus t_{15}$
1	$t_3 = x_0 \oplus x_1$	2	$g_1 = \text{NAND}(x_2, t_1)$	3	$t_{13} = g_5 \odot g_1$	4	$y_2 = g_7 \oplus g_8$
1	$t_4 = x_0 \oplus x_5$	2	$g_2 = \text{NAND}(x_2, t_4)$	3	$t_{14} = g_1 \odot g_4$	4	$y_3 = t_{13} \odot g_9$
1	$t_5 = x_1 \odot x_3$	2	$g_3 = \text{NAND}(t_2, x_4)$	3	$t_{15} = g_2 \odot g_6$	4	$y_4 = t_{14} \odot g_9$
1	$t_6 = x_3 \oplus x_4$	2	$g_4 = \text{NAND}(t_4, t_5)$	3	$t_{16} = g_2 \odot g_3$	4	$y_5 = t_{12} \odot g_{10}$
2	$t_7 = t_2 \oplus t_3$	2	$g_5 = \text{OR}(x_5, t_0)$	3	$g_8 = \text{NAND}(x_1, t_7)$		
2	$t_8 = x_0 \oplus t_2$	2	$g_6 = \text{OR}(x_5, t_3)$	3	$g_9 = \text{NAND}(t_9, t_6)$		